

# **1 Chapter 1: Introduction**

## **1.1 1.1 Background**

Smart energy monitoring systems are increasingly built from connected sensing devices, lightweight communication protocols, local or cloud gateways, time-series storage, and visualization dashboards. These systems help users and operators observe voltage, current, power, and device status over time. However, monitoring is more useful when the system can also identify abnormal conditions quickly, not only store raw measurements for later review.

IoT-based energy monitoring commonly uses microcontrollers and network modules to publish measurements from distributed devices. MQTT is suitable for this kind of system because it supports lightweight publish/subscribe messaging between devices and backend services. An edge gateway can subscribe to MQTT topics, validate incoming payloads, detect important events, and store the resulting data in a time-series database for dashboards and later analysis.

## **1.2 1.2 Problem Statement**

Raw telemetry pipelines can store energy measurements but often provide limited local intelligence. If telemetry is only collected and stored, abnormal conditions such as overload, voltage deviation, device silence, or malformed payloads may not become visible until a later dashboard query or manual review. As the number of devices grows, the monitoring pipeline also needs observable validation, event classification, and latency evidence.

This thesis addresses that problem by designing and implementing an event-driven edge gateway for IoT-based smart energy monitoring. The gateway uses MQTT for device communication, validates incoming payloads, applies rule-based event detection, stores time-series readings in TimescaleDB, and exposes readings, events, and system metrics through Grafana dashboards.

## **1.3 1.3 Motivation**

The motivation for this work is the need for timely, observable, and extensible smart energy monitoring. A raw telemetry pipeline can show what happened, but it may not immediately classify abnormal conditions or expose data-quality problems. In practical energy monitoring, operators need to know whether devices are reporting, whether measurements look abnormal, and whether the gateway is processing messages within acceptable latency.

Academically, the project is useful because it provides a measurable architecture comparison. Instead of only building a dashboard, this thesis compares a baseline ingestion path against a proposed event-driven edge gateway. The comparison uses throughput, latency, validation, event-detection, and observability evidence.

## 1.4 1.4 Aim and Objectives

The aim of this thesis is to design and implement an event-driven edge gateway for IoT-based smart energy monitoring using MQTT, rule-based event detection, TimescaleDB storage, and Grafana observability dashboards.

The specific objectives are:

1. Build an MQTT-based telemetry pipeline for smart energy monitoring devices.
2. Implement an edge gateway that validates incoming telemetry and status messages.
3. Detect abnormal energy conditions using configurable rule-based processing.
4. Store accepted readings, events, validation logs, and operational metrics in PostgreSQL/TimescaleDB.
5. Provide Grafana dashboards for energy overview, device detail, event timeline, system observability, and thesis evaluation.
6. Compare a baseline ingestion path with the proposed event-driven gateway using throughput, latency, validation, event-detection, and storage-cost evidence.
7. Add an unsupervised edge anomaly detector (Isolation Forest) and evaluate it against rule-based detection (Phase 1 of a hybrid edge–cloud direction).

## 1.5 1.5 Scope of the Project

The scope of the project is the design, implementation, and evaluation of the smart energy monitoring pipeline from MQTT telemetry ingestion to edge processing, storage, and dashboards.

Included:

- STM32-based energy node
- MQTT communication
- edge gateway processing
- rule-based event detection
- edge ML anomaly detection (Isolation Forest, Phase 1), offline-evaluated
- PostgreSQL/TimescaleDB storage
- Grafana dashboards

Excluded / out of scope:

- full production deployment
- large-scale field hardware validation
- cloud-tier ML, forecasting, and score-gated edge→cloud escalation
- certified commercial metering accuracy
- storage reduction as a measured result

## 1.6 1.6 Research Questions

This thesis answers the following research questions:

1. How can an event-driven edge gateway improve smart energy monitoring pipelines?
2. How does the proposed architecture compare with a baseline telemetry-storage approach?
3. Can rule-based processing detect abnormal energy events while maintaining low processing latency?
4. Can a lightweight unsupervised model (Isolation Forest) add anomaly detection at the edge, and how does it compare with rule-based detection?

## 1.7 1.7 Expected Contributions

The expected contributions are:

1. A working edge-gateway architecture for IoT smart energy monitoring.
2. A baseline versus proposed evaluation showing the overhead of event-driven edge processing.
3. Rule-based detection evidence for overload, power spike, voltage anomaly, under-voltage, over-voltage, and device-failure conditions.
4. Validation and data-quality evidence for malformed MQTT payloads.
5. Grafana dashboards that expose readings, events, gateway metrics, and thesis evaluation results.
6. An edge Isolation Forest anomaly detector (Phase 1) with an offline precision/recall evaluation and an honest rules-versus-ML comparison.
7. A hybrid edge-cloud extension of the architecture: a score-gated edge-to-cloud escalation path (Phase 2) and a cloud-tier LSTM autoencoder verifier (Phase 3), each with a controlled measurement, plus a clearly defined path toward cloud-tier elastic scaling on Kubernetes (Phase 4) — the latter identified as future work rather than claimed as a completed result.

## 1.8 1.8 Thesis Organization

Chapter 1 introduces the problem, motivation, scope, research questions, and expected contributions. Chapter 2 reviews related work on IoT energy monitoring, MQTT communication, edge processing, time-series storage, and observability dashboards. Chapter 3 presents the proposed system architecture. Chapter 4 describes the implementation methodology, including the gateway, database, rules, and dashboards. Chapter 5 explains the experimental evaluation method. Chapter 6 presents and discusses the results. Chapter 7 concludes the thesis and describes future work such as cloud-tier elastic scaling on Kubernetes, storage optimization, and broader hardware validation.

# 2 Chapter 2: Literature Review

## 2.1 2.1 Smart Energy Monitoring Systems

Smart energy monitoring systems are designed to measure electrical parameters such as voltage, current, power, and energy consumption, then communicate those measure-

ments to users or operators. A typical system contains a sensing layer, communication layer, processing layer, storage layer, and visualization layer. The sensing layer measures electrical behavior. The communication layer transmits measurements. The processing layer validates or analyzes data. The storage layer keeps historical records. The visualization layer presents readings, events, and trends through dashboards or applications.

Smart meters and energy monitoring systems are important because they provide near-real-time visibility into consumption and power quality. Smart metering literature commonly emphasizes two-way communication, remote monitoring, and fine-grained measurement as improvements over manual meter reading or periodic billing-only systems [1]. Low-cost IoT energy monitoring prototypes also show that microcontrollers and wireless communication can reduce manual effort and provide users with more immediate energy-consumption feedback [2].

However, many basic monitoring systems focus on collecting and displaying readings. They may not measure the cost of adding local event intelligence, or compare a raw telemetry pipeline against an edge-processing pipeline. This thesis follows the layered smart energy monitoring model but focuses on the edge gateway layer, where validation, event classification, and metrics can be performed before long-term storage.

## **2.2 2.2 IoT-Based Energy Data Acquisition**

IoT-based energy acquisition systems commonly use microcontrollers, electrical sensors, and wireless modules. The device samples voltage and current, calculates power, packages the values into a payload, and transmits the payload through a communication protocol. Low-cost systems often use Arduino, ESP, STM32, optical meter pulse sensors, current transformers, or voltage sensing circuits depending on the required accuracy and safety constraints [2].

In a practical deployment, hardware design must consider sensor calibration, electrical isolation, sampling frequency, firmware timing, network reliability, and power supply stability. These concerns are important for field deployment, but they are different from the gateway evaluation performed in this thesis. The current evaluation uses a simulator to produce repeatable MQTT telemetry. This allows the software pipeline to be tested under controlled high-throughput and anomaly scenarios without mixing hardware measurement error into the gateway analysis.

The simulator-based approach is appropriate for evaluating ingestion latency, rule-processing overhead, validation behavior, and dashboard observability. Real STM32/ESP hardware validation remains future work.

## **2.3 2.3 MQTT in IoT Communication**

MQTT is widely used in IoT systems because it is a lightweight publish/subscribe protocol. MQTT.org describes it as an OASIS standard messaging protocol for IoT, designed for remote devices with small code footprints and limited bandwidth [3]. Its

publish/subscribe model separates publishers from subscribers through a broker. Devices publish messages to topics, and interested consumers subscribe to topic filters.

This model is suitable for distributed energy monitoring. Energy nodes do not need to know the address or implementation details of the gateway. They only publish telemetry to a topic. The gateway subscribes to the relevant topic pattern and processes messages as they arrive. MQTT also supports bidirectional communication, quality-of-service levels, persistent sessions, and TLS-based security options [3], [4].

Research on MQTT-based wireless sensor nodes also highlights its simplicity, low overhead, and suitability for constrained networked devices compared with heavier request/response protocols [5]. Other MQTT research explores broker extensions and distributed broker architectures, showing that MQTT is common enough in IoT systems that researchers study how to add filtering, aggregation, or edge-oriented broker behavior [6], [7].

This thesis uses the following MQTT topic structure:

```
energy/{device_id}/telemetry
energy/{device_id}/status
energy/{device_id}/events
```

The edge gateway subscribes to:

```
energy/+/telemetry
energy/+/status
energy/+/events
```

This topic design separates measurements, device status, and events while remaining simple enough for low-cost devices or simulators.

## 2.4 2.4 Edge and Fog Computing for IoT Monitoring

Cloud-only IoT architectures can centralize storage and processing, but they may introduce latency, bandwidth cost, and dependency on wide-area network availability. Edge and fog computing address this by placing computation and storage closer to the data source. Surveys of fog computing describe fog nodes as intermediate computation and storage resources between IoT devices and cloud systems, useful for latency-sensitive applications [8]. Edge/fog computing literature also notes that processing IoT data closer to devices can reduce cloud dependence and support faster application response [8], [9].

For smart energy monitoring, this means a gateway can validate and classify telemetry before data is sent to long-term storage or cloud systems. This is important because abnormal conditions such as overload, under-voltage, or device silence may require timely attention. Waiting until a later dashboard query may delay awareness.

This thesis applies the edge-computing idea through a FastAPI gateway. The gateway runs near the MQTT broker and database in the local stack. It consumes messages, validates payloads, applies rule-based detection, stores events, and exposes metrics. The

evaluation then measures the latency overhead of that edge processing compared with a simpler baseline ingestion path.

## 2.5 2.5 Time-Series Storage and Observability

Energy readings are naturally time-series data. Each reading is meaningful because of its timestamp, device ID, and measured values. Common queries include latest value per device, readings over a time range, aggregated power over time, event counts by minute, validation failures, and gateway latency history. These query patterns fit a time-series storage model.

TimescaleDB extends PostgreSQL for time-series workloads. Its documentation describes hypertables as PostgreSQL tables that automatically partition time-series data by time and optionally by other dimensions, allowing queries to target relevant chunks instead of scanning an entire table [10]. This is appropriate for energy monitoring because recent readings and time-window queries are central to the dashboard and evaluation workflow.

Grafana is commonly used for observability dashboards. The Grafana PostgreSQL datasource supports time-series queries, table queries, template variables, annotations, alerting, and built-in time filtering/grouping macros [11]. This makes it suitable for visualizing readings, events, validation failures, and gateway metrics stored in PostgreSQL/TimescaleDB.

In this project, TimescaleDB stores `energy_readings`, `events`, `data_quality_logs`, `device_status_history`, and `system_metrics`. Grafana then visualizes these tables through energy overview, device detail, event timeline, system observability, and thesis evaluation dashboards.

## 2.6 2.6 Rule-Based Event Detection

Monitoring systems often begin with rule-based event detection. A rule-based system defines abnormal conditions using thresholds, comparisons, or state-change logic. Examples include voltage below a lower bound, current above an overload threshold, or power increasing by a percentage within a short window.

Rule-based detection has several advantages for a thesis prototype. It is explainable, deterministic, simple to configure, and fast to evaluate at the edge. It is also easier to defend in an experimental comparison because each event can be traced back to a specific condition and threshold.

Rule-based detection also has limitations. Fixed thresholds may not adapt to seasonal behavior, different household patterns, equipment age, or changing loads. More advanced anomaly detection literature often studies statistical or machine-learning methods for IoT and energy systems [12], [13]. Those methods can be useful, but they require training data, model evaluation, and careful treatment of false positives and false negatives.

Rule-based detection is the starting point of this thesis and remains the baseline detector. Building on it, the work is extended with an unsupervised machine-learning detector at the edge (Section 2.7), so that rule-based and model-based detection can be compared directly rather than treating ML purely as future work.

## 2.7 2.7 Hybrid Edge–Cloud AI for Energy Monitoring

A recent line of work moves beyond rule-based monitoring toward placing *lightweight machine learning at the edge* and reserving *heavier models for a cloud tier*. Four studies in particular motivate the direction taken here.

Verde Romero et al. [14] present an open-source IoT edge-computing system for building energy monitoring (Mosquito MQTT, Node-RED, SQLite) across an asset–edge–cloud hierarchy. Their central argument is that edge/fog processing reduces latency, improves privacy, and preserves operation during internet outages relative to a cloud-only design. They demonstrate real-time dashboards but do not quantify latency, and they explicitly name *exploring artificial intelligence at the edge* as future work. This thesis treats that stated future work as a concrete objective: the edge gateway here is a stronger substrate (FastAPI, TimescaleDB, a configurable rule engine, durable alerting, and a quantitative A/B harness) on which edge AI can be implemented and measured.

Mofidul et al. [15] build a secured IIoT infrastructure with ESP32-S nodes (ZMPT101B voltage and SEN0211 current sensors), MQTTS/TLS, PostgreSQL, and **per-appliance Isolation Forest** anomaly detection on edge and cloud ( $n\_estimators = 100$ ,  $contamination = 0.01$ ). Their work supports three choices made here: Isolation Forest as an unsupervised, label-free edge detector; TLS-secured MQTT and per-device credentials as a security direction; and a sensor bill of materials that matches the intended hardware node. They also show that *per-device* models outperform a single model over heterogeneous data — a finding that frames the per-device extension as the natural next step beyond the global model used in this phase.

Sathupadi et al. [16] propose an edge–cloud synergy in which a lightweight detector runs at the edge and forwards only flagged windows to a heavier cloud model, governed by an anomaly score threshold ( $score > \mu + \alpha \cdot \sigma$ ) and a dynamic workload manager balancing latency, bandwidth, and energy. This threshold-gated escalation pattern is the blueprint for the hybrid architecture targeted by this thesis: cheap detection at the edge, selective escalation to the cloud, measured on latency and bandwidth.

Huang et al. [17] describe a four-layer edge/fog/cloud framework for user-side energy management that adds LSTM-attention load forecasting, demand response, and notably **edge-side data compression** (lossless Huffman/LZW and lossy DCT/PCA), reporting substantial per-node data-volume reductions. This supplies concrete techniques and evaluation metrics (compression ratio, information retention) for the storage-optimization phase, which the current thesis otherwise leaves as an unproven claim.

Taken together, these works converge on a single pattern — *rules/lightweight ML at the edge, heavier ML in the cloud, gated by an anomaly score, evaluated on latency,*

*bandwidth, and storage* — which this thesis adopts and begins to realise. Phase 1 (this document set) implements and offline-evaluates the edge Isolation Forest detector; the cloud tier, escalation gate, and storage optimization are staged as subsequent phases.

## 2.8 2.8 Review of Related Work

The related literature and technical references show that smart energy monitoring, MQTT-based communication, edge/fog computing, time-series storage, and anomaly detection are established research and engineering areas. The gap is not the existence of any one component. The gap is the measured integration of these components into a baseline-versus-proposed edge-gateway evaluation.

| Work                                         | Focus                                                        | Technology                           | Strengths                                                      | Limitations                                                            | Relation to this thesis                                              |
|----------------------------------------------|--------------------------------------------------------------|--------------------------------------|----------------------------------------------------------------|------------------------------------------------------------------------|----------------------------------------------------------------------|
| Smart meter and AMI literature [1]           | Remote energy measurement and two-way metering communication | Smart meters, AMI networks           | Establishes the importance of near-real-time energy visibility | Utility-scale systems are broader than this prototype                  | Provides background for energy monitoring requirements               |
| Islam et al. low-cost smart energy meter [2] | Low-cost IoT energy monitoring                               | Arduino, optical sensor, Android app | Shows practical low-cost monitoring and user feedback          | Focuses more on meter reading/application than edge-gateway evaluation | Motivates low-cost IoT monitoring but leaves gateway comparison open |
| MQTT official/OASIS material [3], [4]        | Lightweight IoT messaging                                    | MQTT broker, clients, topics, QoS    | Supports constrained devices and publish/subscribe decoupling  | Security and ordering require careful system design                    | Justifies MQTT topic-based communication in this project             |
| MQTT wireless sensor node work [5]           | MQTT for sensor networks                                     | MQTT, Wi-Fi, embedded nodes          | Shows MQTT suitability for simple networked sensing            | Not specific to energy event detection                                 | Supports the communication choice                                    |



| Work                                                | Focus                                                    | Technology                                 | Strengths                                                  | Limitations                                                             | Relation to this thesis                                          |
|-----------------------------------------------------|----------------------------------------------------------|--------------------------------------------|------------------------------------------------------------|-------------------------------------------------------------------------|------------------------------------------------------------------|
| MQTT+ and distributed MQTT broker research [6], [7] | Broker-side filtering, aggregation, and distributed MQTT | MQTT extensions and broker architectures   | Shows interest in processing closer to MQTT infrastructure | Changes or extends broker behavior rather than gateway-layer processing | Reinforces the value of edge-side processing near MQTT data flow |
| Fog computing survey [8]                            | Processing between IoT devices and cloud                 | Fog nodes, edge resources                  | Explains latency-aware local computation for IoT           | Broad survey, not energy-specific                                       | Supports the edge-gateway architectural decision                 |
| Edge/fog/cloud overview [9]                         | Distributed computation for IoT                          | Edge, fog, cloud                           | Discusses latency and bandwidth motivation                 | Broad application scope                                                 | Frames why local gateway processing is useful                    |
| TimescaleDB documentation [10]                      | Time-series storage                                      | PostgreSQL hypertables and chunks          | Supports time-window queries and partitioning              | Storage optimization must still be designed per workload                | Justifies TimescaleDB for readings/events/metrics                |
| Grafana PostgreSQL documentation [11]               | Dashboard observability                                  | PostgreSQL datasource, macros, annotations | Supports time-series visualization and table dashboards    | Dashboard quality depends on query design                               | Justifies Grafana for thesis evidence and operational visibility |
| IoT anomaly detection survey material [12]          | Anomaly detection methods for IoT                        | Statistical and ML methods                 | Shows importance of anomaly detection in IoT systems       | ML methods require datasets and model evaluation                        | Positions ML as future work beyond rule-based detection          |

| Work                                    | Focus                                            | Technology                                                   | Strengths                                                                 | Limitations                                                 | Relation to this thesis                                                          |
|-----------------------------------------|--------------------------------------------------|--------------------------------------------------------------|---------------------------------------------------------------------------|-------------------------------------------------------------|----------------------------------------------------------------------------------|
| IoT smart energy management review [13] | Smart energy analytics and energy disaggregation | IoT networks, energy management, algorithms                  | Shows broader direction toward intelligent analysis                       | Focuses on advanced analytics beyond this implementation    | Supports future ML/forecasting extension direction                               |
| Verde Romero et al. [14]                | Open-source edge energy monitoring for buildings | Mosquitto, Node-RED, SQLite, as-set/edge/cloud               | Demonstrates edge/fog benefits (latency, privacy, offline operation)      | No quantified latency; edge AI only proposed as future work | Primary motivation; this thesis implements and measures the edge AI they propose |
| Mofidul et al. [15]                     | Secured IIoT with edge/cloud AI                  | ESP32-S, MQTTS/TLS, Post-greSQL, Isolation Forest            | Real hardware, per-appliance IF, strong security                          | Per-device models; field-specific                           | Justifies Isolation Forest, TLS/MQTT security, and sensor BOM                    |
| Sathupadi et al. [16]                   | Edge–cloud synergy for anomaly detection         | Edge detector + cloud model, score-gated escalation          | Threshold-gated escalation; workload manager for latency/bandwidth/energy | Synthetic/industry data; light on rigorous ground truth     | Blueprint for the hybrid escalation architecture                                 |
| Huang et al. [17]                       | User-side energy management framework            | Edge/fog/cloud LSTM forecasting, DCT/PCA/Huffman compression | Concrete compression techniques and metrics                               | Broad scope; some results illustrative                      | Supplies storage/bandwidth optimization techniques and metrics                   |

## 2.9 Research Gap

Existing work shows that smart energy monitoring systems can collect and display measurements, MQTT can support lightweight IoT messaging, edge/fog computing can reduce cloud dependence, time-series databases can support time-window queries, and anomaly detection is important for IoT systems. However, many systems do not provide

a measured comparison between a raw telemetry ingestion pipeline and an event-driven edge gateway using latency, throughput, validation behavior, event-detection counts, and dashboard observability evidence.

This thesis responds to that gap by implementing and evaluating a complete edge-gateway pipeline:

1. MQTT topic-based telemetry, status, and event ingestion.
2. Payload validation and data-quality logging.
3. Rule-based event detection at the gateway.
4. PostgreSQL/TimescaleDB storage for readings, events, validation logs, and metrics.
5. Grafana dashboards for operational and thesis evidence.
6. A clean baseline-versus-proposed high-throughput comparison.
7. A separate proposed-mode anomaly detection experiment.

The contribution is therefore not only a smart meter, not only a dashboard, and not an ML anomaly detector. The contribution is a measured edge-gateway architecture for smart energy monitoring, showing that rule-based event intelligence can be added with low latency overhead.

## 2.10 References

- [1] V. C. Gungor et al., “Survey of Smart Metering Communication Technologies,” IEEE Communications Surveys & Tutorials, 2011.
- [2] M. R. Islam, S. Sarker, M. S. Mazumder, and M. R. Ranim, “An IoT based Real-time Low Cost Smart Energy Meter Monitoring System using Android Application,” arXiv:2001.10350, 2020.
- [3] MQTT.org, “MQTT: The Standard for IoT Messaging.” <https://mqtt.org/>
- [4] OASIS, “MQTT Version 5.0,” OASIS Standard, 2019. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/>
- [5] Z. Shao, M. Huang, D. Wu, X. Zhang, and A. Huang, “Design of a Simplified Wireless Sensor Network Node based on MQTT Protocol,” arXiv:1906.10540, 2019.
- [6] R. Giambona, A. E. C. Redondi, and M. Cesana, “MQTT+: Enhanced Syntax and Broker Functionalities for Data Filtering, Processing and Aggregation,” arXiv:1810.00773, 2018.
- [7] E. Longo, A. E. C. Redondi, M. Cesana, A. Arcia-Moret, and P. Manzoni, “MQTT-ST: a Spanning Tree Protocol for Distributed MQTT Brokers,” arXiv:1911.07622, 2019.
- [8] R. K. Naha et al., “Fog Computing: Survey of Trends, Architectures, Requirements, and Research Directions,” arXiv:1807.00976, 2018.
- [9] T. Vo, P. Dave, G. Bajpai, and R. Kashef, “Edge, Fog, and Cloud Computing: An Overview on Challenges and Applications,” arXiv:2211.01863, 2022.

- [10] Tiger Data, “Hypertables,” TimescaleDB documentation. <https://www.tigerdata.com/docs/use-timescale/latest/hypertables>
- [11] Grafana Labs, “PostgreSQL data source,” Grafana documentation. <https://grafana.com/docs/grafana/latest/datasources/postgresql/>
- [12] A. Chatterjee and B. S. Ahmed, “IoT anomaly detection methods and applications: A survey,” *Internet of Things*, 2022.
- [13] G. Huang, A. Anwar, S. W. Loke, A. Zaslavsky, and J. Choi, “IoT-based Analysis for Smart Energy Management,” *arXiv:2311.18643*, 2023.
- [14] D. A. Verde Romero, E. Villalvazo Laureano, R. O. Jiménez Betancourt, and E. Navarro Álvarez, “An open source IoT edge-computing system for monitoring energy consumption in buildings,” *Results in Engineering*, vol. 21, 101875, 2024.
- [15] R. B. Mofidul, M. M. Alam, M. H. Rahman, and Y. M. Jang, “Real-Time Energy Data Acquisition, Anomaly Detection, and Monitoring System: Implementation of a Secured, Robust, and Integrated Global IIoT Infrastructure with Edge and Cloud AI,” *Sensors*, vol. 22, no. 22, 8980, 2022.
- [16] K. Sathupadi, S. Achar, S. V. Bhaskaran, N. Faruqui, M. Abdullah-Al-Wadud, and J. Uddin, “Edge-Cloud Synergy for AI-Enhanced Sensor Network Data: A Real-Time Predictive Maintenance Framework,” *Sensors*, vol. 24, no. 24, 7918, 2024.
- [17] J. Huang, S. Zhou, G. Li, and Q. Shen, “Real-time monitoring and optimization methods for user-side energy management based on edge computing,” *Scientific Reports*, vol. 15, 24890, 2025.

## 3 Chapter 3: Proposed System Architecture

### 3.1 System Overview

The proposed system is an edge-first, event-driven architecture for IoT-based smart energy monitoring. Energy monitoring nodes publish measurements through MQTT. A local broker receives those messages and forwards them to a FastAPI edge gateway. The gateway validates the payloads, applies rule-based event detection, classifies abnormal conditions, stores readings and events in PostgreSQL/TimescaleDB, and exposes the data through REST APIs and Grafana dashboards.

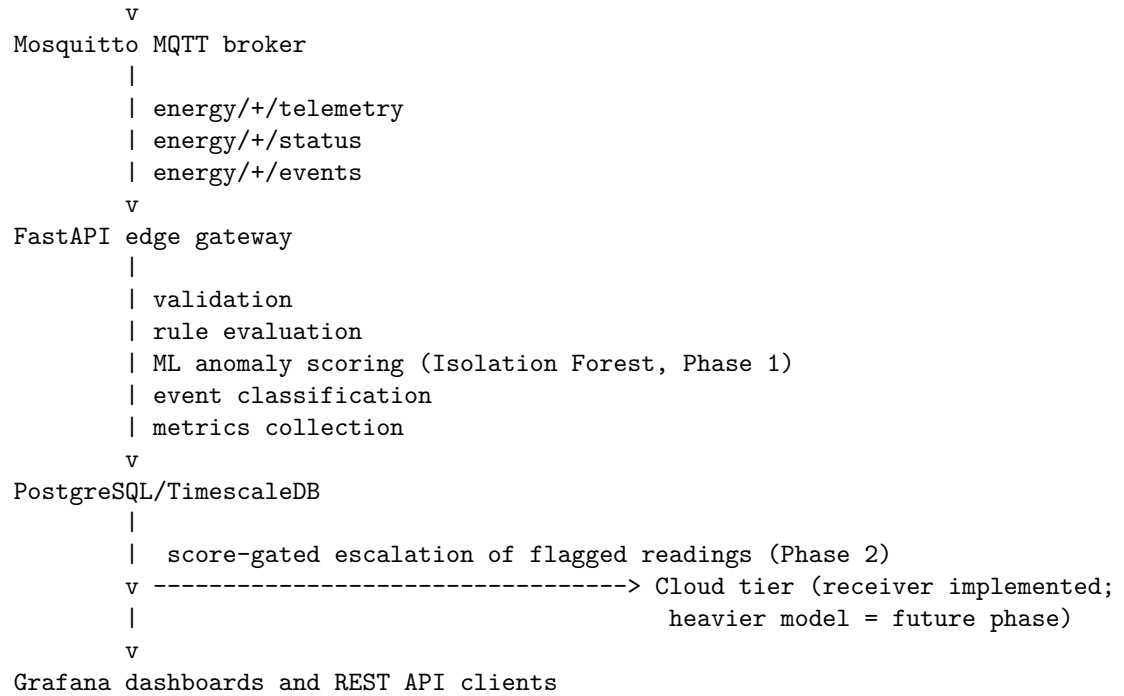
The main architectural contribution is the edge gateway layer. Instead of treating the backend as only a database writer, the gateway acts as a local intelligence layer between energy devices and long-term storage. This allows the system to reject malformed data, detect abnormal conditions early, and record event evidence before the information is viewed on a dashboard.

The high-level flow is:

```

Energy node or simulator
|
| MQTT telemetry/status/events

```



The architecture follows five principles:

1. Process telemetry at the edge before treating it as stored history.
2. Separate normal readings from abnormal events.
3. Store time-series data in a database designed for time-window queries.
4. Make validation, latency, throughput, and event counts observable.
5. Add edge intelligence incrementally — rule-based detection first, then a lightweight ML detector at the edge (Phase 1), then score-gated escalation to a cloud tier (Phase 2), then a cloud-side LSTM-autoencoder verifier on the escalated stream (Phase 3).

## 3.2 3.2 Energy Monitoring Node Layer

The energy monitoring node is responsible for measuring electrical parameters and publishing structured telemetry. In the intended physical setup, this layer can be implemented using an STM32 microcontroller with voltage and current sensing hardware and an ESP8266 or ESP32 module for Wi-Fi/MQTT communication.

The node is expected to measure:

- voltage in volts
- current in amperes
- instantaneous power in watts
- optional temperature
- device status and signal metadata

The basic local calculation is:

```
power_w = voltage_v * current_a
```

Each node uses a stable device identifier such as `house_0001` or `lab_node_001`. The device identifier appears both in the MQTT topic and in the message body, allowing the gateway to associate readings, status messages, and events with the correct source.

For the current thesis evaluation, synthetic devices are produced by the MQTT simulator. This allows repeatable high-throughput and anomaly tests without depending on field hardware timing. Real STM32/ESP hardware can be connected to the same MQTT topic structure in future hardware validation.

### 3.3 MQTT Broker Layer

The MQTT broker decouples publishers from consumers. Energy nodes publish messages to topic names, and the edge gateway subscribes to topic patterns. This allows devices and backend services to evolve independently.

The broker used in the implementation is Eclipse Mosquitto. The topic design separates telemetry, status, and events:

```
energy/{device_id}/telemetry
energy/{device_id}/status
energy/{device_id}/events
energy/{device_id}/commands
energy/{device_id}/config
```

The gateway subscribes to:

```
energy+/telemetry
energy+/status
energy+/events
```

This topic structure is simple enough for a prototype but still expressive enough to distinguish measurement data, device health, and device-originated events. It also allows the gateway to route messages to different validation and ingestion paths based on topic type.

### 3.4 FastAPI Edge Gateway

The FastAPI edge gateway is the main contribution of the thesis. It combines an MQTT consumer, validation pipeline, rule engine, persistence layer, metrics collector, alert workflow, and REST API.

The gateway responsibilities are:

| Responsibility       | Description                                                                                                         |
|----------------------|---------------------------------------------------------------------------------------------------------------------|
| MQTT consumption     | Subscribe to energy topics and receive telemetry, status, and event messages                                        |
| Topic parsing        | Extract device ID and message type from <code>energy/{device_id}/{type}</code>                                      |
| Payload validation   | Validate telemetry/status/event schemas before storage                                                              |
| Data-quality logging | Record invalid payloads and validation failure reasons                                                              |
| Rule evaluation      | Apply threshold and percentage-increase rules to valid telemetry                                                    |
| ML anomaly scoring   | Score each reading with an Isolation Forest detector (Phase 1) and persist scores to <code>model_predictions</code> |
| Event classification | Convert rule hits and ML anomalies into typed events with severity                                                  |
| Persistence          | Store devices, readings, events, status history, metrics, and quality logs                                          |
| Metrics              | Track counters and latency summaries for evaluation                                                                 |
| REST API             | Expose health, readings, devices, events, rules, and metrics endpoints                                              |
| Alert workflow       | Queue critical-event notifications through an alert outbox                                                          |

The gateway is implemented as a modular service. MQTT handlers, ingestion services, repositories, schemas, rule evaluation, metrics, and background workers are separated into different modules. This separation makes the gateway easier to test and easier to extend.

### 3.5 3.5 Data Validation Pipeline

The validation pipeline protects the database and dashboards from malformed or incomplete input. Each incoming MQTT message is routed by topic type and validated against the expected schema.

Telemetry payloads include:

| Field                  | Meaning                           |
|------------------------|-----------------------------------|
| <code>device_id</code> | Stable device identifier          |
| <code>timestamp</code> | Device-side timestamp             |
| <code>voltage_v</code> | RMS voltage in volts              |
| <code>current_a</code> | RMS current in amperes            |
| <code>power_w</code>   | Instantaneous real power in watts |

| Field            | Meaning                            |
|------------------|------------------------------------|
| temperature_c    | Optional temperature               |
| sequence_no      | Optional monotonic sequence number |
| firmware_version | Optional firmware version          |
| rsssi_dbm        | Optional Wi-Fi signal strength     |

Status payloads include online/offline/maintenance/error status and optional metadata such as firmware version, IP address, RSSI, and reason.

When validation succeeds, telemetry can be stored and evaluated by the rule engine. When validation fails, the gateway does not silently discard the problem. It increments validation counters and writes a row to `data_quality_logs`. This makes data quality part of the observable system.

### 3.6 Rule Engine and Event Classifier

The rule engine converts raw readings into meaningful events. Rules are configured in YAML and loaded by the gateway. The current rules include threshold-based checks and percentage-increase checks.

Implemented rule examples:

| Rule                | Condition                       | Event type       | Severity |
|---------------------|---------------------------------|------------------|----------|
| undervoltage        | voltage_v < 200                 | UNDER_VOLTAGE    | WARNING  |
| overvoltage         | voltage_v > 250                 | OVER_VOLTAGE     | WARNING  |
| overload            | current_a > 10                  | OVERLOAD         | CRITICAL |
| power_spike         | power_w increases by 30% in 60s | POWER_SPIKE      | WARNING  |
| voltage_anomaly_low | voltage_v < 210                 | VOLTAGE_ANOMALY  | INFO     |
| over_temperature    | temperature_c > 60              | OVER_TEMPERATURE | WARNING  |

When a rule triggers, the gateway creates an event containing:

- event type
- severity
- rule name
- message
- event value
- threshold value
- related device



- related reading time
- metadata

This event model allows the dashboard and API to explain not only that an event occurred, but also why it occurred.

### 3.7 PostgreSQL/TimescaleDB Storage Layer

The storage layer uses PostgreSQL with TimescaleDB features for time-series data. TimescaleDB is appropriate because most queries are time-window queries: latest readings, readings in a period, per-device trends, event timelines, and system metrics over time.

The main tables are:

| Table                 | Purpose                                                              |
|-----------------------|----------------------------------------------------------------------|
| devices               | Current device metadata and status                                   |
| energy_readings       | Time-series telemetry readings                                       |
| events                | Rule-generated and device-originated events                          |
| data_quality_logs     | Validation errors and malformed payload evidence                     |
| device_status_history | Device status timeline                                               |
| system_metrics        | Gateway counters and latency summaries                               |
| rule_definitions      | Future database-backed rule management                               |
| alert_outbox          | Pending or processed alert notifications                             |
| alert_deliveries      | Alert delivery history                                               |
| model_predictions     | Edge ML anomaly scores and labels (Phase 1); extensible to forecasts |

The important indexes support device/time lookup and event filtering. The time-series tables can be converted into TimescaleDB hypertables, allowing efficient time-window queries and future retention policies. The architecture also supports aggregated views such as one-minute reading summaries.

Storage reduction is not claimed as a measured result in this thesis version. The current implementation stores raw readings and event evidence so the evaluation can compare behavior clearly. Future work may add selective raw retention, downsampling, or event-only long-term storage.

### 3.8 Grafana Dashboard and Alerting

Grafana provides the visual observability layer. The dashboards are provisioned from JSON files and connected to TimescaleDB through a configured PostgreSQL datasource.

The implemented dashboard set is:

| Dashboard            | Purpose                                                                                        |
|----------------------|------------------------------------------------------------------------------------------------|
| Energy Overview      | System-level view of devices, readings, load, voltage, and events                              |
| Device Detail        | Per-device measurements, thresholds, event markers, and status history                         |
| Event Timeline       | Recent events, severity counts, event types, and affected devices                              |
| System Observability | Message rate, validation rate, latency percentiles, and alert queue state                      |
| Thesis Evaluation    | A/B result summary, anomaly evidence, database-size observation, and interpretation boundaries |

The alert workflow is represented through an alert outbox. Critical events can be queued for delivery through configured channels such as webhook, email, or Slack-style integrations. The outbox pattern separates event creation from notification delivery so that detection is not blocked by a temporary external notification failure.

### 3.9 3.9 Edge ML Anomaly Detection (Phase 1)

The architecture is extended with an unsupervised machine-learning detector that runs at the edge alongside the rule engine. The detector is an **Isolation Forest** trained offline on normal operating data and loaded by the gateway as a model artifact (following Mofidul et al. [15]; see Chapter 2.7).

The detector scores a physics-informed feature vector derived from each reading:

```
[ voltage_v, current_a, power_w, temperature_c,
  |voltage_v - V_nominal|,          # voltage excursion
  power_w - voltage_v*current_a ]   # P vs implied apparent power
```

It returns an anomaly score (higher = more anomalous) and flags the reading when the score exceeds a threshold derived from the training distribution. The score and label are written to `model_predictions` for every reading; when event emission is enabled, a flagged reading also raises an `ML_ANOMALY` event through the same storage and alert path as a rule hit.

To keep machine-learning inference off the ingestion hot path, scoring runs in an asynchronous micro-batch worker: the telemetry handler enqueues each reading and returns immediately, and the worker scores readings in batches. This is both a latency optimization (scikit-learn is far cheaper per reading when scoring a batch than one sample at a time) and a structural stepping stone toward the score-gated cloud escalation of a later phase, which consumes the same queue.

The detector is independent of the rule engine, which lets the evaluation run three configurations — **rules-only**, **ml-only**, and **hybrid** — and compare detection quality and

processing overhead directly. It also degrades gracefully: if ML is disabled or the artifact is unavailable, the gateway behaves exactly as the rule-based system, so the baseline path carries no ML dependency.

This realises the edge-AI direction that Verde Romero et al. [14] named as future work.

**Score-gated escalation (Phase 2).** The scoring worker’s queue doubles as the substrate for the hybrid design’s escalation path (following the edge-gates-cloud pattern of Sathupadi et al. [16]): scored readings are offered to a cloud forwarder that, depending on `CLOUD_FORWARD_MODE`, forwards nothing (`off`, the default), only readings whose anomaly score crosses the escalation threshold (`gated`), or every scored reading (`all` — the naive all-to-cloud baseline used for the bandwidth comparison). Forwarded readings are batched into compact JSON envelopes carrying the measurements plus score, threshold, and model version, and POSTed to a minimal cloud-tier receiver that counts what it receives. Forward failures are counted and dropped rather than retried, so edge detection never depends on cloud reachability. The remaining heavier cloud-side model on the escalated stream is added in Phase 3 (Section 6.7.3): an LSTM autoencoder that confirms or suppresses escalated readings by reconstruction error. Edge-side storage optimization (Huang et al. [17]) remains staged as a later phase and is not claimed as a result here.

### 3.10 Chapter Summary

The proposed architecture combines MQTT communication, an event-driven edge gateway, TimescaleDB storage, rule-based event detection, alert workflow, and Grafana dashboards. The key design decision is to move validation and event detection into the gateway layer before data becomes only historical storage. This enables the evaluation in later chapters: the baseline path measures raw ingestion behavior, while the proposed path measures the cost and value of adding edge intelligence.

## 4 Chapter 4: Implementation Methodology

### 4.1 Implementation Overview

This chapter describes how the proposed architecture was implemented. The implementation uses Docker Compose to run Mosquitto, TimescaleDB, the FastAPI edge gateway, Grafana, and the MQTT simulator. The gateway code is organized around schemas, MQTT handlers, ingestion services, repositories, rule evaluation, metrics collection, and background workers.

The current implementation and evaluation focus on the software pipeline. The architecture supports STM32/ESP-based hardware nodes, but the repeatable evaluation in Chapter 5 uses a synthetic MQTT simulator to generate controlled high-throughput and anomaly scenarios.

## 4.2 Hardware and Device Setup

The intended hardware node contains:

- STM32 microcontroller
- voltage sensing circuit
- current sensing circuit
- ESP8266 or ESP32 network module
- power supply
- optional temperature sensor

The device layer is responsible for sampling electrical measurements, calculating power, creating a timestamped payload, and publishing it through MQTT. A practical hardware implementation can publish to the same topic structure used by the simulator.

For controlled thesis evaluation, the simulator replaces physical devices. It generates multiple virtual devices, publishes telemetry and status messages, and can inject abnormal values or malformed payloads. This gives repeatable test data for throughput, latency, validation, and rule-detection experiments.

## 4.3 Firmware and Publisher Workflow

The firmware or simulator publisher follows the same logical workflow:

1. Read voltage and current measurements.
2. Calculate instantaneous power.
3. Optionally read temperature and signal strength.
4. Build a JSON payload with device ID and timestamp.
5. Publish telemetry to `energy/{device_id}/telemetry`.
6. Publish status or heartbeat messages to `energy/{device_id}/status`.
7. Optionally publish device-originated events to `energy/{device_id}/events`.

The gateway expects telemetry fields such as:

```
{
  "device_id": "house_0001",
  "timestamp": "2026-06-28T10:00:00+00:00",
  "voltage_v": 220.5,
  "current_a": 4.2,
  "power_w": 926.1,
  "temperature_c": 32.4,
  "sequence_no": 1024,
  "firmware_version": "0.1.0",
  "rssi_dbm": -55.0
}
```

The exact physical sampling method is outside the measured scope of this software-focused evaluation. The important implementation contract is that hardware or simulated publishers produce the same MQTT topic and payload format.

## 4.4 4.4 MQTT Topic and Payload Design

The MQTT topic convention is:

energy/{device\_id}/{message\_type}

The implemented message types are:

| Topic                        | Purpose                                                 |
|------------------------------|---------------------------------------------------------|
| energy/{device_id}/telemetry | Voltage, current, power, temperature, and sequence data |
| energy/{device_id}/status    | Device health, firmware, IP address, RSSI, and reason   |
| energy/{device_id}/events    | Optional events published by the device                 |

The gateway subscribes to wildcard topics:

energy+/telemetry

energy+/status

energy+/events

Topic parsing is implemented in the MQTT topic helper. Invalid topic shapes are ignored or rejected before they reach the ingestion path. This prevents unstructured MQTT messages from being stored as valid energy telemetry.

## 4.5 4.5 Backend API and MQTT Consumer

The backend is implemented with FastAPI. It starts the MQTT consumer and background workers during application lifespan startup. The MQTT consumer receives broker messages and passes them to the appropriate handler based on topic type.

The REST API exposes:

| API group | Examples                                              | Purpose                                   |
|-----------|-------------------------------------------------------|-------------------------------------------|
| Health    | /health, /ready, /version                             | Service and dependency status             |
| Devices   | /api/v1/devices, /api/v1/devices/{device_id}/history  | Device metadata and status                |
| Readings  | /api/v1/readings, /api/v1/readings/{device_id}/latest | Telemetry query and latest reading access |
| Events    | /api/v1/events, /api/v1/events/{event_id}/knowledge   | Event timeline and knowledge              |
| Rules     | /api/v1/rules, /api/v1/rules/reload                   | Rule inspection and reload                |

| API group | Examples                                                                        | Purpose                                                 |
|-----------|---------------------------------------------------------------------------------|---------------------------------------------------------|
| Metrics   | <code>/api/v1/metrics/summary</code><br><code>/api/v1/metrics/throughput</code> | Gateway counters, latency, quality, and event summaries |

The API supports both dashboard usage and result export. The evaluation scripts use the metrics and report export paths to generate the evidence used in Chapter 6.

## 4.6 Validation and Ingestion Implementation

Incoming payloads are validated with Pydantic schemas before persistence. Telemetry, status, and device-originated event messages have separate schema types. The telemetry schema requires non-negative voltage, current, and power values and accepts optional temperature, sequence number, firmware version, and RSSI.

The ingestion path performs these steps:

1. Parse the MQTT topic.
2. Decode the JSON payload.
3. Validate the payload against the expected schema.
4. Upsert or update device metadata.
5. Store valid readings or status history.
6. Evaluate telemetry against rules when proposed processing is enabled.
7. Store generated events.
8. Record counters and latency metrics.

If payload parsing or validation fails, the gateway increments validation failure counters and writes a `data_quality_logs` record containing the topic, error type, error message, and raw payload. This behavior is important because invalid data becomes measurable evidence instead of disappearing silently.

## 4.7 Database Schema and Persistence

The database schema is managed through Alembic migrations under `database/migrations`. The schema contains operational tables, time-series tables, and future extension tables.

The main tables are:

| Table                              | Implementation purpose                          |
|------------------------------------|-------------------------------------------------|
| <code>devices</code>               | Current device state and metadata               |
| <code>energy_readings</code>       | Valid telemetry readings                        |
| <code>events</code>                | Rule-generated and device-originated events     |
| <code>data_quality_logs</code>     | Invalid payload and validation failure evidence |
| <code>device_status_history</code> | Device online/offline/status history            |

| Table             | Implementation purpose                                               |
|-------------------|----------------------------------------------------------------------|
| system_metrics    | Persisted counters and latency summaries                             |
| rule_definitions  | Future path for database/API-driven rule management                  |
| alert_outbox      | Pending alert notifications                                          |
| alert_deliveries  | Delivery result history                                              |
| model_predictions | Edge ML anomaly scores and labels (Phase 1); extensible to forecasts |

The `energy_readings` table uses `(time, device_id)` as its primary key and has an index on `(device_id, time)` for device time-series queries. The `events` table stores severity, rule name, event value, threshold value, and acknowledgement state. The `system_metrics` table stores gateway counters and latency summaries that are later visualized in Grafana.

TimescaleDB is used because the dominant queries are time-range queries: recent readings, per-device trends, event timelines, and metric history. Hypertables and continuous aggregates provide a future path for efficient longer-term time-series analysis.

## 4.8 Rule Engine Implementation

Rules are configured in `gateway/config/rules.yaml` and loaded by the rule engine at gateway startup. The implemented rule engine supports:

- threshold rules
- percentage-increase rules
- reload through the rules API
- rolling window state for spike detection
- event severity and event type mapping

Example rules include:

| Rule                | Type                | Condition                                 | Event                     |
|---------------------|---------------------|-------------------------------------------|---------------------------|
| undervoltage        | Threshold           | <code>voltage_v &lt; 200</code>           | UNDER_VOLTAGE, warning    |
| overvoltage         | Threshold           | <code>voltage_v &gt; 250</code>           | OVER_VOLTAGE, warning     |
| overload            | Threshold           | <code>current_a &gt; 10</code>            | OVERLOAD, critical        |
| power_spike         | Percentage increase | <code>power_w</code> rises 30% within 60s | POWER_SPIKE, warning      |
| voltage_anomaly_low | Threshold           | <code>voltage_v &lt; 210</code>           | VOLTAGE_ANOMALY, info     |
| over_temperature    | Threshold           | <code>temperature_c &gt; 60</code>        | OVER_TEMPERATURE, warning |

When a rule triggers, the engine returns a rule hit. The ingestion service turns that hit into a stored event with a device ID, event type, severity, rule name, message, event value, threshold value, and metadata. This provides traceability between raw telemetry and the generated event.

## 4.9 4.9 Edge ML Anomaly Detector Implementation

The edge ML detector is implemented in `gateway/app/services/anomaly_detector.py` and trained by `scripts/train_anomaly_model.py`. It uses an Isolation Forest from scikit-learn, persisted as a joblib artifact and mounted read-only into the gateway container at `/app/models`.

**Feature engineering (physics\_v1).** Each reading is mapped to a six-element vector: the four raw fields [`voltage_v`, `current_a`, `power_w`, `temperature_c`] plus two physics-informed features — the voltage excursion `|voltage_v - 220|` and the power consistency residual `power_w - voltage_v*current_a`. The same transform is implemented in both the training script (`engineer()`) and the gateway (`_apply_engineering`) under the `physics_v1` tag. These features are needed because a global Isolation Forest over raw whole-house load dilutes single-axis anomalies across its random splits and does not isolate out-of-range values quickly; the engineered features make voltage excursions and voltage/current/power inconsistencies separable.

**Training and threshold.** The training script generates data that mirrors the simulator’s telemetry distribution, including the simulator’s behaviour of overriding only the anomaly field on top of an otherwise-normal reading. It fits a `StandardScaler` and an `IsolationForest` (`n_estimators = 200`, `max_samples = 1024`, `contamination = 0.01`), then defines the anomaly score as `-model.score_samples(x)` so that higher means more anomalous. The detection threshold is the  $q$ -th quantile of the normal-data scores (default  $q = 0.90$ ), which exposes an explicit recall/false-positive operating point.

**Scoring in the pipeline.** When `ENABLE_ML=true`, each valid reading is scored, the score and label are written to `model_predictions`, the `ml.scored/ml.anomalies` counters are updated, and (when `ML_EMIT_EVENTS=true`) a flagged reading raises an `ML_ANOMALY` event through the same storage and alert path as a rule hit, with a per-device cooldown to bound event volume. The detector is independent of the rule engine, enabling rules-only, ml-only, and hybrid configurations. If ML is disabled or the artifact is missing, the detector disables itself and the gateway runs exactly as the rule-based system, so the baseline path carries no ML dependency.

**Async micro-batch scoring.** The first online A/B (Section 6.7.1) showed that calling scikit-learn’s `score_samples` once per message, synchronously inside the ingestion path, added roughly 12 ms of latency per reading — scikit-learn is optimised for batched inference, so per-call overhead dominates single-sample scoring (profiling measured ~10 ms per single sample versus ~0.014 ms per row when batched). To remove this from the hot path, scoring was moved into an asynchronous micro-batch worker (`gateway/app/workers/ml_scoring.py`). The telemetry handler enqueues a lightweight job per reading and returns immediately; the worker drains the queue



in batches (bounded by `ML_BATCH_MAX_SIZE` or `ML_BATCH_WINDOW_MS`), scores each batch in a single model call, and persists predictions and ML events. This keeps telemetry latency at rule-engine levels while amortising inference cost across the batch. The `ML_ASYNC_SCORING` flag (default true) switches between this worker and the original inline path, which allows the inline-versus-batched comparison in Section 6.7.1.

## 4.10 4.10 Edge-to-Cloud Escalation Gate (Phase 2)

The hybrid architecture’s second phase adds a score-gated escalation path from the edge gateway to a cloud tier, implemented in `gateway/app/workers/cloud_forwarder.py` and a minimal receiver service in `cloud/app/main.py`.

**Escalation gate.** After the ML scoring worker scores a batch, each scored reading is offered to the cloud forwarder, which applies the gate configured by `CLOUD_FORWARD_MODE`:

- `off` (default) — nothing leaves the edge; the gateway behaves exactly as in Phase 1.
- `gated` — only readings whose anomaly score crosses the escalation threshold are forwarded. By default the gate reuses the model’s own anomaly threshold (escalating exactly the flagged set); `CLOUD_ESCALATION_THRESHOLD` can set a stricter score cut-off independently of event emission.
- `all` — every scored reading is forwarded. This is the naive all-data-to-cloud baseline: both modes run the identical pipeline, so a gated-versus-all comparison isolates the gate as the only variable.

**Forwarding worker.** Escalated readings are queued and sent in batches (bounded by `CLOUD_FORWARD_BATCH_MAX_SIZE` or `CLOUD_FORWARD_BATCH_WINDOW_MS`) as a compact JSON envelope over HTTP POST. Each forwarded reading carries the measurement fields plus its anomaly score, threshold, model version, and whether a rule also fired — enough context for a cloud-side model to act without a follow-up query. The gateway counts forwarded readings, batches, and payload bytes (`cloud.forwarded`, `cloud.batches`, `cloud.bytes_sent`) and records a `cloud_forward` latency series. Failures are counted (`cloud.forward_failed`) and the batch is dropped rather than retried: edge detection never depends on cloud reachability, preserving the edge-first principle.

**Cloud tier.** The receiver (`cloud-tier` in Docker Compose) is deliberately minimal for the Phase 2 bandwidth measurement: it validates the envelope, counts batches, readings, and received bytes, and keeps a bounded in-memory buffer of recent escalations for inspection. Its role there is to terminate the escalation path so bandwidth can be measured end to end (`scripts/run_escalation_bandwidth_test.sh`, Section 5.6). Phase 3 extends the same service with an LSTM-autoencoder verifier (`cloud/app/verifier.py`) that scores escalated readings by reconstruction error using a numpy-only kernel, so the container still needs no deep-learning runtime (Section 6.7.3).

## 4.11 4.11 Metrics and Alert Workflow

The metrics service keeps in-memory counters and latency samples, then periodically flushes them to `system_metrics`. The exported metrics include message counters, validation failures, readings stored, event counts, and latency percentiles.

Latency summaries are reported as:

- average
- p50
- p95
- p99

These metrics are used by the evaluation scripts and Grafana dashboards. They allow the thesis to compare baseline and proposed modes using measured gateway behavior rather than only simulator output.

The alert workflow uses an outbox pattern. When critical events are created, alert records can be queued in `alert_outbox`. A background worker can process pending alerts and record delivery results. This separates detection from delivery and prevents a notification failure from blocking event persistence.

## 4.12 4.12 Dashboard Implementation

Grafana is provisioned from files under `config/grafana`. The datasource is configured to connect to TimescaleDB, and dashboard JSON files are loaded into the Energy Monitoring folder.

The implemented dashboards are:

| Dashboard                              | Implementation role                                                            |
|----------------------------------------|--------------------------------------------------------------------------------|
| <code>energy_overview.json</code>      | System-level readings, load, voltage, and event summary                        |
| <code>device_detail.json</code>        | Per-device status, readings, thresholds, events, and status history            |
| <code>event_timeline.json</code>       | Event table, event buckets, severity summary, affected devices                 |
| <code>system_observability.json</code> | Message rate, stored-reading rate, validation rate, latency, alert outbox      |
| <code>thesis_evaluation.json</code>    | A/B summary, latency overhead, anomaly evidence, and interpretation boundaries |

The dashboards are designed to support both operation and thesis evidence. They do not replace the exported result files, but they visually confirm that readings, events, data-quality behavior, and system metrics are observable.

## 4.13 4.13 Docker-Based Deployment

Docker Compose is used to run the local system. The main services are:

| Service      | Container role                    | Exposed port                    |
|--------------|-----------------------------------|---------------------------------|
| mosquitto    | MQTT broker                       | 18831 -> 1883                   |
| timescaledb  | PostgreSQL/TimescaleDB database   | 54329 -> 5432                   |
| edge-gateway | FastAPI gateway and MQTT consumer | 8001 -> 8000                    |
| grafana      | Dashboard UI                      | 3001 -> 3000                    |
| simulator    | MQTT workload generator           | profile-based load-test service |

The gateway is configured through environment variables such as `PROCESSING_MODE`, `STORE_RAW_READINGS`, `ENABLE_RULE_ENGINE`, `ENABLE_AGGREGATION`, `ENABLE_ALERTS`, `ENABLE_ML`, and `ML_EMIT_EVENTS`. This makes it possible to switch between baseline and proposed modes and between rules-only, ml-only, and hybrid detection for evaluation. The trained model artifact is mounted read-only at `/app/models/anomaly_iforest.joblib`.

## 4.14 4.14 Testing Strategy

The testing strategy combines automated tests and scripted system experiments.

Automated tests cover:

- MQTT topic parsing
- payload validation
- rule engine behavior
- ML anomaly detector behavior (disabled path, missing-artifact path, live scoring)
- repository operations
- metrics service behavior
- alert service and outbox worker behavior
- API smoke checks
- script syntax checks
- application lifecycle behavior

Scripted experiments cover:

- baseline high-throughput ingestion
- proposed high-throughput processing
- repeated clean A/B comparisons
- proposed-mode anomaly scenarios
- invalid payload handling
- report and snapshot export

The key scripts are:

| Script                                              | Purpose                                                |
|-----------------------------------------------------|--------------------------------------------------------|
| <code>scripts/run_baseline_test.sh</code>           | Single baseline experiment                             |
| <code>scripts/run_proposed_test.sh</code>           | Single proposed experiment                             |
| <code>scripts/run_high_throughput_ab_test.sh</code> | Claim repeated baseline/proposed comparison            |
| <code>scripts/run_anomaly_detection_test.sh</code>  | Proposed-mode anomaly and validation evidence          |
| <code>scripts/train_anomaly_model.py</code>         | Train + offline-evaluate the Isolation Forest detector |
| <code>scripts/run_detection_ab_test.sh</code>       | Rules-only vs ml-only vs hybrid detection A/B          |
| <code>scripts/export_results.py</code>              | Export metrics snapshots and Markdown reports          |

This strategy keeps the thesis evaluation reproducible. The automated tests verify the implementation pieces, while the experiment scripts generate the evidence used in Chapter 6.

## 4.15 4.15 Chapter Summary

The implementation follows the proposed architecture closely. MQTT messages enter through Mosquitto, the FastAPI gateway validates and processes them, applies rule-based detection and an edge Isolation Forest anomaly detector, TimescaleDB stores readings, events, and model predictions, Grafana visualizes the system state, and scripts export reproducible evaluation evidence. The implementation now includes a trained, offline-evaluated edge ML detector (Phase 1), with explicit extension points for the cloud tier, score-gated escalation, and storage optimization in later phases.

# 5 Chapter 5: Experimental Evaluation

## 5.1 5.1 Experimental Setup

This chapter evaluates the implemented smart energy monitoring pipeline using a local reproducible test environment. The evaluated system consists of an MQTT broker, a FastAPI edge gateway, PostgreSQL/TimescaleDB storage, Grafana dashboards, and a synthetic MQTT device simulator.

The evaluation focuses on the software pipeline and gateway behavior. The simulator represents multiple energy monitoring devices that publish voltage, current, power, temperature, status, and invalid-payload test messages through MQTT. The experiment therefore measures gateway ingestion, validation, rule-based event detection, database persistence, and observability behavior. It does not claim certified electrical metering accuracy or large-scale field hardware validation.

The stack was executed through Docker Compose. The gateway consumed MQTT messages from Mosquitto, validated incoming payloads, stored accepted telemetry in TimescaleDB, and exported metrics and reports for comparison. Grafana was used to verify dashboard visibility for readings, events, and system metrics.

Before interpreting the benchmark results, the automated test suite was run:

| Test command                                                                               | Result    |
|--------------------------------------------------------------------------------------------|-----------|
| UV_CACHE_DIR=/tmp/uv-cache<br>uv run pytest<br>gateway/tests/scripts/test_scripts.py<br>-q | 2 passed  |
| UV_CACHE_DIR=/tmp/uv-cache<br>uv run pytest gateway/tests<br>-q                            | 45 passed |

These tests verified the script smoke tests and the current gateway behavior covered by the repository test suite.

## 5.2 5.2 Baseline and Proposed System Definition

The evaluation compares two gateway modes: a baseline raw-ingestion mode and a proposed event-driven edge-processing mode.

The baseline mode represents a simpler telemetry pipeline. In this mode, valid MQTT telemetry is validated and stored, but the proposed event-driven rule processing behavior is not used as the main contribution. This provides a reference point for measuring ingestion throughput and latency when the gateway acts mainly as a storage path for valid readings.

The proposed mode represents the thesis architecture. In this mode, the gateway performs validation, applies rule-based detection, classifies events, stores telemetry and event records, and exposes operational metrics for dashboards and thesis evaluation. This mode is intended to show whether an edge gateway can add useful event intelligence without introducing excessive processing latency.

The defensible thesis claim is:

This project designs and implements an event-driven edge gateway for IoT-based smart energy monitoring using MQTT, rule-based event detection, TimescaleDB storage, and Grafana observability dashboards.

Building on this baseline, the work adds an edge machine-learning detector (Phase 1, Section 5.5), so detection can be compared across rules-only, ml-only, and hybrid configurations, and a score-gated edge-to-cloud escalation path (Phase 2, Section 5.6), so gated forwarding can be compared against a naive all-to-cloud baseline. The current work does not yet claim a cloud-side model, storage reduction, production-grade field

deployment, certified metering accuracy, or large-scale smart-grid validation. These are treated as later phases and future work.

### 5.3 5.3 High-Throughput A/B Test Method

The first experiment was a clean high-throughput A/B comparison. The same simulated workload was run in baseline and proposed modes. The purpose was to measure whether the proposed event-driven gateway introduces significant throughput or latency overhead compared with the baseline ingestion path.

The experiment was executed with:

```
REPETITIONS=3 just ab-high-throughput
```

The high-throughput scenario used:

| Parameter         | Value                               |
|-------------------|-------------------------------------|
| Scenario          | high_throughput.yaml                |
| Simulated devices | 200                                 |
| Device interval   | 1.0 second                          |
| Run duration      | approximately 120 seconds           |
| Repetitions       | 3 baseline runs and 3 proposed runs |
| Output directory  | results/ab/high_throughput/         |

The A/B script reset the Docker stack and database volumes between runs. This was important because it prevented previous data from contaminating the counter, event, and database-size measurements.

For each run, the simulator log recorded total messages sent, publish failures, and simulator-side message rate. The gateway exported a report and raw snapshot containing counters and latency summaries.

### 5.4 5.4 Anomaly Detection Experiment Method

The second experiment evaluated proposed-mode rule-based event detection. This experiment was not a baseline/proposed throughput comparison. Instead, it tested whether the proposed gateway could detect abnormal electrical conditions and validation failures under controlled synthetic scenarios.

The experiment was executed with:

```
just anomaly-detection
```

The anomaly detection experiment ran these proposed-mode scenarios:

| Scenario          | Purpose                                                   |
|-------------------|-----------------------------------------------------------|
| undervoltage_test | Verify under-voltage and voltage-anomaly event generation |
| overload_test     | Verify critical overload detection                        |
| power_spike_test  | Verify warning-level power-spike detection                |
| invalid_payloads  | Verify malformed payload rejection and validation metrics |

The output was written under:

`results/anomaly_detection/proposed/`

This experiment collected event counts by type and severity, validation errors by type, gateway counters, and latency summaries.

## 5.5 5.5 Edge ML Detection Evaluation Method (Phase 1)

The edge Isolation Forest detector is evaluated in two complementary ways: an offline detection-quality measurement and an online operational A/B.

**Offline detection quality.** The model is trained and evaluated by `scripts/train_anomaly_model.py --evaluate`. Training data mirrors the simulator's telemetry distribution; a held-out labeled test set combines normal readings with injected anomalies generated the way the simulator generates them (overriding only the anomaly field). Because the labels are known by construction, this yields genuine precision, recall, F1, false-positive rate, per-anomaly-type recall, and an operating-point tradeoff table (detection threshold versus recall/FPR). This separates *detection quality* from the live pipeline and is fully reproducible from a fixed seed without a running stack.

**Online detection A/B.** The script `scripts/run_detection_ab_test.sh` runs the same labeled anomaly scenarios in three gateway configurations:

| Mode   | Configuration                                                       |
|--------|---------------------------------------------------------------------|
| rules  | ENABLE_RULE_ENGINE=true,<br>ENABLE_ML=false                         |
| ml     | ENABLE_RULE_ENGINE=false,<br>ENABLE_ML=true,<br>ML_EMIT_EVENTS=true |
| hybrid | ENABLE_RULE_ENGINE=true,<br>ENABLE_ML=true,<br>ML_EMIT_EVENTS=true  |

For each mode it captures detected events by type, prediction counts and score statistics from `model_predictions`, telemetry and `ml_inference` latency, and database

growth, under:

```
results/detection_ab/{rules,ml,hybrid}/
```

This measures the *operational cost* of adding ML scoring to the live pipeline, complementing the offline detection-quality measurement.

## 5.6 5.6 Escalation Bandwidth A/B Method (Phase 2)

The score-gated edge-to-cloud escalation (Section 4.10) is evaluated by `scripts/run_escalation_bandwidth_test.` which runs the same labeled anomaly scenarios in two gateway configurations that differ *only* in the escalation gate:

| Mode  | Configuration                                                                                         |
|-------|-------------------------------------------------------------------------------------------------------|
| gated | CLOUD_FORWARD_MODE=gated — forward only readings whose anomaly score crosses the escalation threshold |
| all   | CLOUD_FORWARD_MODE=all — forward every scored reading (naive all-to-cloud baseline)                   |

Both modes run the full hybrid pipeline (rules + async ML scoring + event emission) and the cloud-tier receiver, so the comparison isolates the gate. For each mode the script captures the gateway’s forwarding counters (`cloud.forwarded`, `cloud.batches`, `cloud.bytes_sent`, `cloud.forward_failed`, `cloud_forward` latency) and the cloud tier’s independent receive counters (readings, batches, bytes), then computes the byte and reading reduction of gated relative to all-to-cloud in `bandwidth-summary.json`, under:

```
results/escalation_bandwidth/{gated,all}/
```

Counting payload bytes on both the sending and receiving side guards against measurement error: the two totals must agree for a run to be valid. The measured quantity is application-payload bytes (the JSON envelope), not wire-level bytes including HTTP and TCP/IP overhead; because batching gives both modes similar per-batch overhead, the payload-byte ratio is the honest comparison statistic. The expected reduction follows from the gate: in gated mode the forwarded volume tracks the model’s flag rate (roughly 7% of readings in the detection A/B scenarios) rather than the full telemetry stream.

## 5.7 5.7 Metrics Collected

The evaluation used both simulator-side and gateway-side metrics.

Simulator-side metrics:

- total MQTT messages sent



- failed publish attempts
- simulator message rate

Gateway-side metrics:

- `messages.received`
- `messages.telemetry`
- `readings.stored`
- `validation.failures`
- `events.critical`
- `events.warning`
- `events.info`
- event counts by type
- validation errors by type
- `ml.scored` and `ml.anomalies` counters (when ML is enabled)
- `model_predictions` counts and anomaly-score statistics
- `telemetry`, `status`, and `ml_inference` latency percentiles

Latency was reported using average, p50, p95, and p99 values. These values were exported from the gateway metrics snapshot and summarized in generated reports. The p95 and p99 values are especially useful because they show whether the gateway remains responsive outside the average case.

Database size was also captured before and after high-throughput runs. In this thesis, database size is treated as a storage-cost and observability discussion, not as a storage-optimization result. Both modes store raw readings, and the proposed mode also stores event evidence. Therefore, the current experiment does not support a storage-reduction claim.

## 5.8 Data Collection Procedure

The high-throughput A/B test and anomaly detection experiment generated machine-readable and human-readable artifacts.

The high-throughput A/B results were stored under:

`results/ab/high_throughput/`

Each run directory contains:

- `simulator.log`
- `snapshot.json`
- `report.md`
- `db-size-before.txt`
- `db-size-after.txt`

The anomaly detection results were stored under:

`results/anomaly_detection/proposed/`

This directory contains:

- per-scenario simulator logs
- metrics-summary.json
- events-by-severity.txt
- events-by-type.txt
- validation-errors-by-type.txt
- report.md
- database-size snapshots

The exported reports were used to build the result tables in Chapter 6. Grafana dashboards were used as visual evidence that readings, events, validation behavior, and system metrics can be inspected during and after the experiments.

## 5.9 5.9 Experimental Limitations

The evaluation has several limitations.

First, the tests use synthetic simulator workloads. The results demonstrate the gateway and data pipeline behavior, but they do not prove performance with real STM32 or ESP-based hardware under field conditions.

Second, the experiments are short local Docker runs. They show repeatable behavior under controlled test conditions, but they do not prove multi-day production reliability or deployment readiness.

Third, the machine-learning detector is evaluated offline on simulator-faithful synthetic data, and in the live pipeline only its operational cost is measured. The offline precision/recall therefore reflects detection quality on controlled data, not field accuracy. The detector is also a single global Isolation Forest; per-device models (which the literature shows can perform better) and the cloud tier with score-gated escalation remain future phases.

Fourth, storage reduction is not measured as a successful result. The proposed mode writes additional event records, so database growth is expected. Selective raw retention, downsampling, and event-only long-term storage remain future work.

Finally, the system is not evaluated for certified electrical metering accuracy, production security, or large-scale smart-grid deployment. The evaluation is intentionally scoped to the event-driven edge gateway and its observable software behavior.

## 6 Chapter 6: Results and Discussion

### 6.1 6.1 Summary of Results

The experimental results support the main thesis claim that the proposed gateway adds event intelligence with only a small latency overhead. In the clean high-throughput A/B test, the proposed gateway sustained almost the same simulator throughput as the baseline. At the same time, proposed mode produced critical and warning events through rule-based edge processing.

The most important result is that the proposed event-driven edge gateway preserved low-millisecond telemetry processing latency while adding validation, rule evaluation, event generation, storage, and dashboard observability.

## 6.2 6.2 Throughput Results

The high-throughput experiment used 200 simulated devices publishing at a 1-second interval for approximately 120 seconds per run. The experiment was repeated three times in each mode. At matched throughput, the proposed gateway adds only a small ingest-latency overhead over the baseline (see fig. 1).

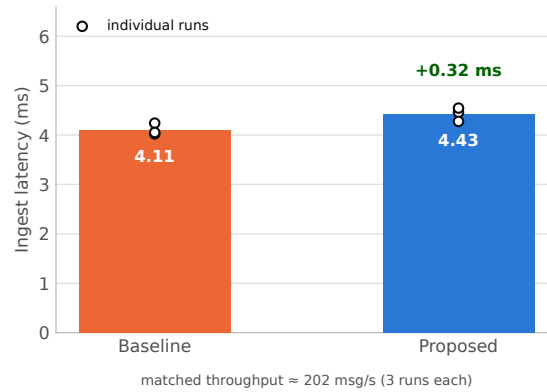


Figure 1: Mean per-message ingest latency, baseline vs. proposed gateway, at matched throughput ( $\approx$ 202 msg/s). Dots are the three individual runs per mode; the proposed gateway adds only +0.32 ms on average.

| Mode                    | Run 1        | Run 2        | Run 3        | Average      |
|-------------------------|--------------|--------------|--------------|--------------|
| Baseline simulator rate | 199.48 msg/s | 199.64 msg/s | 199.71 msg/s | 199.61 msg/s |
| Proposed simulator rate | 199.44 msg/s | 199.57 msg/s | 199.78 msg/s | 199.60 msg/s |

The simulator completed every run with zero publish failures. The average rate was nearly identical between modes: 199.61 messages per second for the baseline and 199.60 messages per second for the proposed gateway. This shows that the proposed rule-processing behavior did not reduce simulator-side throughput under the tested workload.

Gateway counters also showed comparable input volume:

| Metric             | Baseline average | Proposed average | Observation                       |
|--------------------|------------------|------------------|-----------------------------------|
| Uptime             | 122.24 s         | 122.25 s         | Comparable run length             |
| messages.received  | 24,731.67        | 24,744.33        | Comparable input volume           |
| messages.telemetry | 23,501.33        | 23,472.67        | Comparable valid telemetry volume |
| readings.stored    | 23,501.33        | 23,472.33        | Near-complete storage parity      |

One caveat is that proposed run 2 had a single-reading mismatch: `messages.telemetry` was 23,479 while `readings.stored` was 23,478. For this reason, the result should be described as near-complete storage parity rather than perfect storage parity across all runs.

### 6.3 Latency Results

Telemetry latency remained low in both modes. The proposed mode introduced a small but measurable overhead because it performed additional rule-based processing and event generation.

| Metric                | Baseline | Proposed | Difference |
|-----------------------|----------|----------|------------|
| Avg telemetry latency | 4.11 ms  | 4.43 ms  | +0.32 ms   |
| p50 telemetry latency | 4.02 ms  | 4.33 ms  | +0.30 ms   |
| p95 telemetry latency | 4.97 ms  | 5.56 ms  | +0.59 ms   |
| p99 telemetry latency | 5.52 ms  | 6.29 ms  | +0.77 ms   |

The p99 telemetry latency stayed below 7 ms in the repeated high-throughput A/B runs. This is important because p99 latency reflects slower cases better than the average. The measured difference indicates that rule evaluation and event processing added less than 1 ms at p99 in this experiment.

Status-message latency was also similar between modes:

| Metric             | Baseline | Proposed | Difference |
|--------------------|----------|----------|------------|
| Avg status latency | 3.84 ms  | 3.87 ms  | +0.03 ms   |
| p99 status latency | 5.31 ms  | 5.19 ms  | -0.12 ms   |

The latency results support the conclusion that the proposed gateway can add edge intelligence while preserving low-latency ingestion behavior.

## 6.4 6.4 Event Detection Results

The proposed gateway generated event records during both the high-throughput A/B test and the separate anomaly detection experiment. In the high-throughput A/B test, proposed mode produced many more events than baseline mode because rule-based processing was active.

| Metric          | Baseline average | Proposed average |
|-----------------|------------------|------------------|
| events.critical | 136.67           | 3,980.67         |
| events.warning  | 0.00             | 5,948.00         |

The separate anomaly detection experiment provides clearer functional evidence for specific rule types. Proposed mode detected overload, power spike, under-voltage, over-voltage, voltage anomaly, and device failure events.

| Event type      | Severity | Observed count |
|-----------------|----------|----------------|
| OVERLOAD        | Critical | 284            |
| DEVICE_FAILURE  | Critical | 12             |
| POWER_SPIKE     | Warning  | 430            |
| UNDER_VOLTAGE   | Warning  | 31             |
| OVER_VOLTAGE    | Warning  | 14             |
| VOLTAGE_ANOMALY | Info     | 31             |

Severity totals from the anomaly experiment were:

| Severity | Count |
|----------|-------|
| Critical | 296   |
| Warning  | 475   |
| Info     | 31    |

These results show that the event-driven gateway can classify abnormal conditions into severity levels. This directly supports the research question about whether rule-based processing can detect abnormal energy events while maintaining low processing latency.

The anomaly experiment also reported telemetry latency:

| Operation | Count | Avg     | p50     | p95     | p99     |
|-----------|-------|---------|---------|---------|---------|
| Telemetry | 1,709 | 6.62 ms | 6.56 ms | 7.98 ms | 8.78 ms |
| Status    | 68    | 5.30 ms | 5.24 ms | 6.17 ms | 6.29 ms |

Even during anomaly scenarios, telemetry p99 latency stayed below 9 ms.

## 6.5 6.5 Validation and Data Quality Results

Validation behavior was visible in both the high-throughput A/B test and the anomaly experiment. The gateway rejected malformed payloads and exposed validation failures as measurable counters and data-quality logs.

Average validation failures in the high-throughput A/B test:

| Mode     | Average validation failures |
|----------|-----------------------------|
| Baseline | 454.67                      |
| Proposed | 478.00                      |

In the proposed anomaly detection experiment, invalid payload handling was measured directly:

| Error type   | Count |
|--------------|-------|
| invalid_json | 148   |

The anomaly run stored all valid telemetry readings:

| Metric             | Value |
|--------------------|-------|
| messages.telemetry | 1,709 |
| readings.stored    | 1,709 |

These results show that the gateway does not silently accept malformed data. Instead, it rejects invalid payloads, records the failure type, and exposes the behavior through reports and dashboards. This is important for a monitoring system because data quality problems must be visible to operators and researchers.

## 6.6 6.6 Dashboard and Observability Results

Grafana dashboards were used to inspect readings, event timelines, device details, system metrics, and thesis evaluation summaries. The dashboard layer does not create the underlying measurements; it makes the gateway behavior visible.

The following dashboard screenshots should be included in the final formatted thesis after capturing them from the clean proposed run:

| Figure     | Dashboard            | Purpose                                                          |
|------------|----------------------|------------------------------------------------------------------|
| Figure 6.1 | Energy Overview      | Shows system-level readings, load, voltage, and event counts     |
| Figure 6.2 | Device Detail        | Shows per-device measurements with thresholds and event context  |
| Figure 6.3 | Event Timeline       | Shows rule-generated event records and severity classification   |
| Figure 6.4 | System Observability | Shows gateway message rate, validation rate, and latency metrics |
| Figure 6.5 | Thesis Evaluation    | Shows the A/B and anomaly evidence summary used in this chapter  |

The implemented dashboards support the following views:

| Dashboard            | Purpose                                                                                                         |
|----------------------|-----------------------------------------------------------------------------------------------------------------|
| Energy Overview      | Shows devices seen, stored readings, latest load, average voltage, events, and latest readings                  |
| Device Detail        | Shows per-device status, freshness, latest values, time-series readings, thresholds, events, and status history |
| Event Timeline       | Shows recent events, event counts by type and severity, affected devices, and unacknowledged events             |
| System Observability | Shows message rate, stored-reading rate, validation-failure rate, latency percentiles, and alert outbox state   |
| Thesis Evaluation    | Summarizes the A/B result, anomaly detection evidence, database-size observation, and interpretation boundaries |

This supports the thesis contribution because the system is not only storing readings; it also exposes operational evidence for evaluation and monitoring. The dashboards help demonstrate event-driven observability, which is part of the proposed architecture.

## 6.7 6.7 Edge ML Anomaly Detection Results (Phase 1)

This section reports the offline detection quality of the edge Isolation Forest detector. These results were produced by `scripts/train_anomaly_model.py --evaluate` (seed 42, 50,000 normal training samples, a held-out test set of 10,000 normal and 2,000 injected anomalies) and written to `results/anomaly_model/offline_evaluation.json`.

At the shipped operating point (detection threshold = 0.90 quantile of the normal-data anomaly scores):

| Metric                        | Value                   |
|-------------------------------|-------------------------|
| Precision                     | 0.612                   |
| Recall                        | 0.780                   |
| F1                            | 0.686                   |
| False-positive rate           | 0.099                   |
| Confusion (tn / fp / fn / tp) | 9010 / 990 / 439 / 1561 |

Recall varied by anomaly type:

| Anomaly type | Count | Recall |
|--------------|-------|--------|
| overvoltage  | 499   | 0.992  |
| power_spike  | 502   | 0.861  |
| undervoltage | 492   | 0.831  |
| overload     | 507   | 0.444  |

The detection threshold is a tunable operating point. Lowering it raises recall at the cost of more false positives:

| Threshold quantile | Threshold | FPR   | Recall |
|--------------------|-----------|-------|--------|
| 0.90               | 0.545     | 0.099 | 0.780  |
| 0.95               | 0.573     | 0.052 | 0.387  |
| 0.975              | 0.602     | 0.025 | 0.242  |
| 0.99               | 0.634     | 0.011 | 0.188  |

The interpretation is twofold. First, the unsupervised detector reliably flags anomalies that leave the normal operating envelope — over-voltage (0.99), power spikes (0.86), and under-voltage (0.83) — without using any labels. Second, `overload` recall is low (0.44), and this is an expected and useful finding rather than a failure: the simulator’s normal evening load already drives current well above the rule’s 10 A overload threshold, so a rule-based overload alarm fires on normal high-load operation, whereas a detector trained on the actual operating distribution correctly treats those readings as normal. This directly informs the rules-versus-ML discussion: rules are decisive for hard bound violations, while the model adds value for envelope and consistency anomalies and avoids the over-sensitivity of a fixed current threshold.



### 6.7.1 Online detection A/B (operational cost)

The online A/B (`scripts/run_detection_ab_test.sh`) ran the same labeled anomaly scenarios (undervoltage, overload, power\_spike) in three live gateway configurations. It measures the *operational cost* of ML scoring, separate from the offline detection-quality result above. The three modes processed a comparable input volume (1,376–1,379 telemetry messages); small differences and the per-mode event counts reflect the random component of the scenarios and so should be read as indicative rather than exact like-for-like. Adding ML leaves the ingest hot path essentially unchanged while the scoring cost moves into the asynchronous queue, off the critical path (see fig. 2).

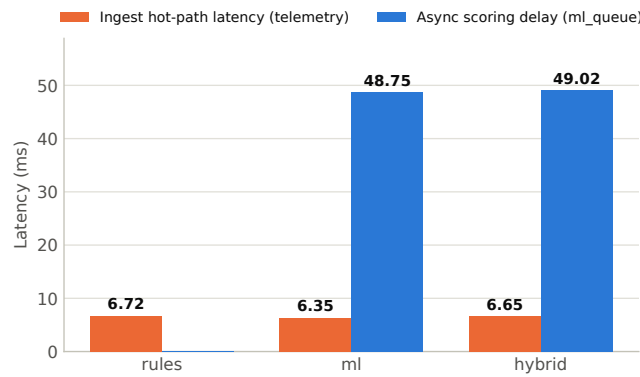


Figure 2: Async scoring decouples ML cost from the ingest hot path. The telemetry hot-path latency stays  $\approx 6.4$ – $6.7$  ms across rules/ml/hybrid, matching rules-only; the scoring cost appears in the asynchronous `ml_queue` delay ( $\approx 49$  ms) instead. This demonstrates decoupling, not batching amortisation.

**Processing latency.** This was the headline operational finding of the first run, in which scoring was performed *inline* (synchronously, one reading at a time) in the ingestion path:

| Mode<br>(inline<br>scoring) | Telemetry avg | Telemetry p99 | ML inference<br>avg | ML inference<br>p99 |
|-----------------------------|---------------|---------------|---------------------|---------------------|
| rules                       | 6.72 ms       | 9.71 ms       | –                   | –                   |
| ml                          | 19.30 ms      | 24.98 ms      | 11.85 ms            | 16.28 ms            |
| hybrid                      | 19.63 ms      | 24.83 ms      | 11.90 ms            | 16.50 ms            |

Inline ML scoring added roughly +12.6 ms average and +15 ms p99 to telemetry processing, dominated by the per-sample `ml_inference` cost ( $\sim 11.9$  ms average). This is much larger than the sub-millisecond overhead of rule evaluation and is an artifact of calling scikit-learn’s `score_samples` on one sample at a time: scikit-learn is optimised for batched inference, so per-message Python/NumPy call overhead dominates

(profiling measured ~10 ms per single sample versus ~0.014 ms per row when scoring a batch). Even so, p99 telemetry latency stayed under 25 ms at the tested throughput.

In response, scoring was moved into an asynchronous micro-batch worker (Section 4.9, `ML_ASYNC_SCORING`, now the default): the telemetry handler enqueues each reading and returns immediately, and the worker scores readings in batches off the hot path. Re-running the A/B with asynchronous scoring enabled gives the “after” measurement:

| Mode<br>(async<br>scoring) | Telemetry avg | Telemetry p99 | ML inference<br>avg / p99 | Queue delay<br>avg / p99 |
|----------------------------|---------------|---------------|---------------------------|--------------------------|
| rules                      | 6.72 ms       | 9.61 ms       | –                         | –                        |
| ml                         | 6.35 ms       | 10.35 ms      | 10.42 / 15.63<br>ms       | 48.75 / 54.97<br>ms      |
| hybrid                     | 6.65 ms       | 9.96 ms       | 10.41 / 13.82<br>ms       | 49.02 / 54.87<br>ms      |

Telemetry processing latency returned to the rule-only baseline (~6.4–6.7 ms average in all three modes): ML scoring no longer adds measurable cost to the ingestion hot path. The cost did not disappear — it moved. The new `ml_queue` series measures the delay from enqueueing a reading to its score being available, ~49 ms average, dominated by the worker’s 50 ms batch-collection window. An `ML_ANOMALY` event therefore trails its reading by roughly the batch window plus inference, still far below any actionable alerting timescale. Two honest caveats: first, at the tested throughput (~2.3 msg/s) the worker collected an average batch of only ~1.1 readings (1,256–1,268 batches for ~1,376 scored), so per-batch inference remains ~10 ms and the amortisation benefit of batching (~0.014 ms per row, Section 4.9) would only materialise at higher message rates — what this experiment demonstrates is the decoupling, not the amortisation. Second, the two runs were taken on the same host but at different times, so small cross-run differences in absolute numbers reflect ordinary load variance; the within-run comparison against the rules mode is the controlled one.

**Detection output.** Counts of generated events and ML scores by mode (from the async-scoring run, which is the evidence retained under `results/detection_ab/`):

| Signal                                                  | rules | ml    | hybrid |
|---------------------------------------------------------|-------|-------|--------|
| Rule events ( <code>OVERLOAD/POWER_SPIKE/...</code> )   | 587   | 0     | 526    |
| <code>ML_ANOMALY</code> events                          | 0     | 6     | 5      |
| <code>model_predictions</code> rows                     | 0     | 1,379 | 1,376  |
| Readings scored anomalous ( <code>ml.anomalies</code> ) | –     | 93    | 88     |

Two points stand out. First, the ml-only mode flagged 93 readings as anomalous but emitted only 6 `ML_ANOMALY` events: the per-device cooldown collapses a burst of anomalous readings during an anomaly window into a single event, which is the intended behaviour for alert hygiene. Second, the hybrid mode produces the union — the full set

of rule events plus a small number of ML events — so a deployment can keep decisive rule alarms for hard bound violations while the model adds coverage. The detection *quality* of these flags (precision/recall) is established offline in Section 6.7, because the live pipeline does not carry per-reading ground-truth labels.

**Storage.** Database growth over each run:

| Mode   | Before   | After    | Growth  |
|--------|----------|----------|---------|
| rules  | 10.01 MB | 11.91 MB | 1.90 MB |
| ml     | 10.01 MB | 12.11 MB | 2.11 MB |
| hybrid | 10.01 MB | 12.55 MB | 2.55 MB |

Hybrid stored the most because it writes rule events, a `model_prediction` row for every reading, and ML events. As in Section 6.8, this confirms that added intelligence has an observable storage cost; it is not a storage-reduction result. Persisting a prediction per reading is the main ML storage contributor and is a candidate for the selective-retention work in a later phase.

### 6.7.2 6.7.2 Escalation bandwidth A/B (Phase 2)

Phase 2 adds the score-gated edge-to-cloud escalation path (Sections 3.9 and 4.10): scored readings whose anomaly score crosses the escalation threshold are batched and forwarded to a minimal cloud-tier receiver, and both sides count forwarded readings and payload bytes. The evaluation compares *gated* forwarding against the naive *all*-to-cloud baseline under the identical pipeline (Section 5.6), so the gate is the only variable.

Both arms ran the same three anomaly scenarios (`undervoltage_test`, `overload_test`, `power_spike_test`) at the same offered load, one run per arm. Raw counters are in `results/escalation_bandwidth/{gated,all}/`, and the reduction is summarised in fig. 3.

| Metric                       | all     | gated   | Reduction |
|------------------------------|---------|---------|-----------|
| Telemetry readings processed | 1,378   | 1,380   | –         |
| Readings forwarded to cloud  | 1,378   | 623     | 54.8%     |
| Escalation batches           | 398     | 316     | 20.6%     |
| Payload bytes sent           | 452,381 | 211,955 | 53.1%     |

**Bandwidth.** Score-gated escalation reduced edge-to-cloud payload volume by 53.1% (452,381 to 211,955 bytes) relative to forwarding every reading, at essentially identical offered load in both arms. Three internal-consistency checks support the number. First, in the gated arm the forwarded count equals the model’s anomaly count exactly (`cloud.forwarded` = `ml.anomalies` = 623): nothing bypasses the gate and no flagged

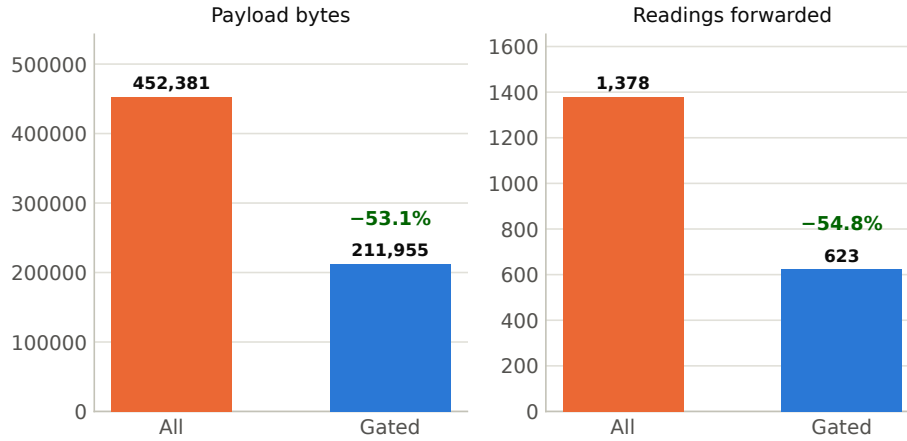


Figure 3: Score-gated vs. all-to-cloud edge→cloud escalation under the same offered load. Gating cut payload volume by 53.1% (bytes) and forwarded readings by 54.8%; receiver-side counters corroborated both volumes.

reading is dropped. Second, the cloud-tier receiver independently counted the same volumes it was sent (623 readings / 211,955 bytes gated; 1,378 readings / 452,381 bytes all), so the sender-side counters are corroborated at the receiving end. Third, the anomaly rate was stable across arms ( $623/1,380 = 45.1\%$  gated,  $633/1,378 = 45.9\%$  all), confirming the two runs saw comparable inputs.

**Edge cost.** Gating added no measurable edge-side latency: telemetry processing averaged 7.09 ms (p95 8.52 ms) gated versus 7.08 ms (p95 8.54 ms) in the all arm, and ML inference and queue latencies matched to within run-to-run noise. The per-batch cloud-forward call itself averaged ~2.1–2.2 ms in both arms.

**Workload dependence.** The 53.1% figure is specific to this deliberately anomaly-heavy workload, in which the model flagged ~45% of readings — far above the ~7% flag rate seen in the mixed detection A/B scenarios (Section 6.7.1). Because gated volume tracks the flag rate by construction, traffic dominated by normal readings would yield a proportionally larger reduction; under this stress mix the measured 53.1% is therefore best read as a conservative bound for this configuration, and no figure is claimed for workloads that were not measured. The gate also has an inherent cost worth stating: the cloud tier never observes non-anomalous readings, which limits cloud-side retraining, drift monitoring, and post-hoc analysis of readings the edge model missed. A periodic sample or summary channel alongside the gated stream is an obvious mitigation and is left as future work (Section 7.5). Finally, these are single runs per arm; the bandwidth counts are near-deterministic given the scenario scripts, but the latency comparison should be read as indicative rather than as a formally repeated measurement.

### 6.7.3 6.7.3 Cloud-tier verification A/B (Phase 3)

Phase 3 adds the heavier cloud-side model motivated by the hybrid direction: an LSTM autoencoder that re-examines the readings the edge escalated. It is trained offline (`scripts/train_cloud_lstm.py`) on normal telemetry windows drawn from the same simulator-mirroring generators as the edge Isolation Forest, so the two tiers see a consistent world. Each reading is embedded in an eight-reading window and scored by its reconstruction error; a reading whose error exceeds a threshold (the 0.95 quantile of normal-window errors) is *confirmed* anomalous, while a well-reconstructed reading is treated as a likely edge false positive. The trained weights are exported to a numpy artifact and the cloud tier runs the forward pass in numpy alone (no deep-learning runtime in the container); a parity check asserts the numpy kernel matches the training model to within  $1.7e-6$  before export.

**Offline detection quality.** On a labeled test set of 8,000 normal and 2,000 anomalous readings, the two-stage detector (edge gate followed by cloud confirmation) is compared against the edge model alone:

| Detector                         | Precision | Recall | F1    | False positives |
|----------------------------------|-----------|--------|-------|-----------------|
| Edge only (Isolation Forest)     | 0.663     | 0.783  | 0.718 | 796             |
| Two-stage (edge → cloud LSTM-AE) | 0.910     | 0.776  | 0.838 | 153             |

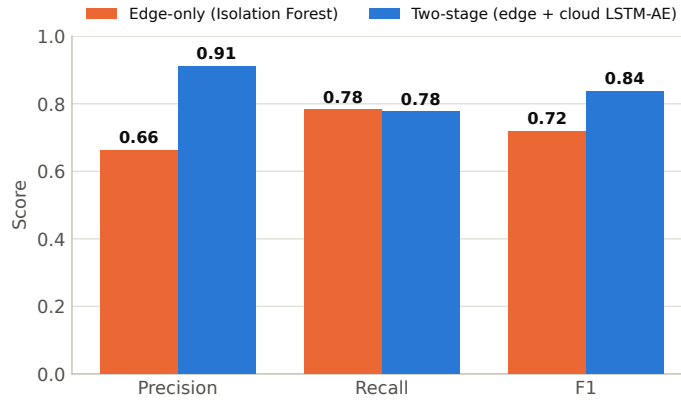


Figure 4: Offline two-stage detection quality on the held-out set (8,000 normal / 2,000 anomalies): the cloud LSTM-AE verifier raises precision from 0.66 to 0.91 at near-identical recall ( $0.78 \rightarrow 0.78$ ), lifting F1 from 0.72 to 0.84.

The cloud confirmation step raised precision from 0.66 to 0.91 by suppressing 643 of the 796 edge false positives, at the cost of dropping only 13 of the 1,565 true anomalies the edge caught (recall 0.783 to 0.776). Per-type recall is essentially unchanged (overvoltage, power-spike, and undervoltage recall identical to the edge; overload down marginally from 0.436 to 0.410), so the precision gain does not come from discarding

a particular anomaly class. This is the central Phase 3 result: a heavier second-opinion model on the *already filtered* stream is a cheap way to trade a small amount of recall for a large precision gain, which directly reduces false alarms without re-processing the full telemetry volume. fig. 5 shows this suppression breakdown.

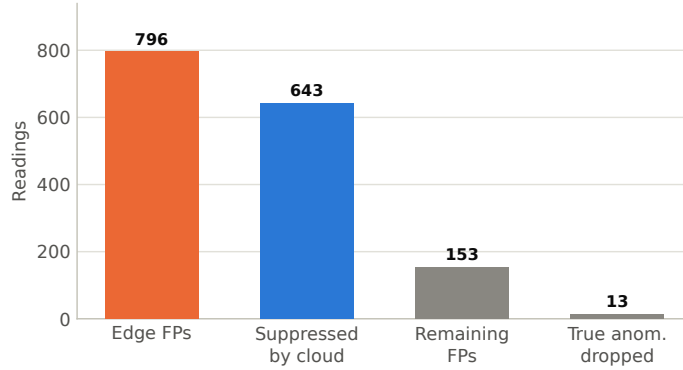


Figure 5: False-positive suppression by the cloud verifier (offline). Of the 796 edge false positives, 643 are suppressed by the cloud stage (leaving 153), at the cost of dropping only 13 true anomalies.

**Online cost and behaviour.** The verifier was then run live in gated mode over the three anomaly scenarios (`results/cloud_verification/`). The escalation chain stayed internally consistent — the gateway forwarded 99 readings (`ml.anomalies = cloud.forwarded = 99`) and the cloud received exactly 99. The verifier scored the ten complete eight-reading windows it accumulated and confirmed all 80 readings, with reconstruction errors of 1.33–661 against the 1.10 threshold: because the online escalations occur during genuine anomaly windows, they are all true anomalies, so a ~100% confirmation rate is the expected and correct outcome, and demonstrates that the verifier loses no true anomalies online. The false-positive-suppression benefit is quantified offline (above) rather than online, because the live gated stream contains few edge false positives to suppress. Cloud inference cost was negligible — 0.65 ms per eight-reading window (~0.08 ms per reading) — so the second tier adds no meaningful latency to the escalation path.

**Caveats.** The autoencoder is trained and evaluated on synthetic, simulator-derived data, so the precision figure characterises the model on this data distribution, not on field telemetry; the window is treated as an unordered batch rather than a true time series (the simulator emits conditionally independent per-tick readings), so the “LSTM” here exploits the joint feature distribution rather than temporal dynamics — a genuine sequential signal from real devices is future work. As with Phase 2 these are single runs, and the 19 escalated readings that did not fill a window were not scored, which is the intended windowing behaviour rather than a loss.

## 6.8 Database Size Discussion

Database size was measured before and after the clean high-throughput A/B runs. The measurements are useful as a storage-cost observation, but they do not prove storage optimization.

| Mode     | Avg size before | Avg size after | Avg growth |
|----------|-----------------|----------------|------------|
| Baseline | ~9.94 MB        | ~21.84 MB      | ~11.89 MB  |
| Proposed | ~9.96 MB        | ~25.90 MB      | ~15.94 MB  |

The proposed mode used more database space. This is expected because proposed mode stores additional event and operational evidence. Therefore, this thesis should not claim storage reduction. A fair storage-reduction claim would require a separate implementation and experiment using selective raw-data retention, downsampling, or event-only long-term storage.

The correct interpretation is that proposed mode has an observable storage cost in exchange for additional event intelligence and monitoring evidence.

## 6.9 Discussion Against Research Questions

The first research question asks how an event-driven edge gateway can improve smart energy monitoring pipelines. The results show that the gateway improves the pipeline by adding validation, rule-based detection, event classification, and dashboard visibility before data is treated only as stored telemetry.

The second research question asks how the proposed architecture compares with a baseline telemetry-storage approach. The high-throughput A/B results show that proposed mode preserved approximately the same simulator throughput as baseline mode while adding event records. The latency overhead was small: average telemetry latency increased by 0.32 ms and p99 latency increased by 0.77 ms.

The third research question asks whether rule-based processing can detect abnormal energy events while maintaining low latency. The anomaly detection experiment confirms that overload, power spike, voltage, and device-failure events were detected, while telemetry p99 latency remained below 9 ms in the anomaly run.

The fourth research question asks how ready the platform is for AI/ML extension. Phase 1 answers this concretely: an edge Isolation Forest detector is implemented, trained offline, and evaluated (Section 6.7), scoring every reading into `model_predictions` and optionally raising `ML_ANOMALY` events through the same path as rules. The platform is therefore no longer only ML-ready in principle; it runs a working edge ML detector, and Phase 2 extends it with a measured score-gated escalation path to a cloud-tier receiver (Section 6.7.2). Cloud-side analytical models and storage optimization remain staged as later phases.

## **6.10 6.10 Limitations**

The results should be interpreted within the scope of the experiment.

First, the workload was generated by a simulator. The results are valid for the implemented software pipeline, but real hardware tests are still needed to evaluate sensor behavior, firmware timing, Wi-Fi reliability, and physical measurement accuracy.

Second, the experiments were short local Docker runs. They do not prove long-term production reliability, cloud deployment readiness, or large-scale field performance.

Third, the machine-learning detection-quality result in Section 6.7 is an offline measurement on simulator-faithful synthetic data, not a field result; the online A/B in Section 6.7.1 measures only operational cost, not live precision/recall. The online A/B also showed that per-sample ML inference adds substantial latency (~12 ms) in the current naive implementation — an optimization target (batched or offloaded scoring) rather than a fundamental limit. The detector is a single global Isolation Forest. Per-device models, adaptive thresholds, forecasting, predictive maintenance, and the cloud tier remain future work.

Fourth, storage reduction was not achieved or measured as a successful result. The proposed mode increased database growth because it stored additional event evidence. Storage optimization remains a separate future experiment.

Finally, the system is not a certified metering platform. It demonstrates an event-driven monitoring architecture and software pipeline, not certified commercial billing accuracy.

## **6.11 6.11 Chapter Conclusion**

The results show that the proposed event-driven gateway adds useful event intelligence with only a small latency overhead. Under the repeated high-throughput workload, the proposed system sustained approximately the same simulator message rate as the baseline. It increased average telemetry latency from 4.11 ms to 4.43 ms and p99 telemetry latency from 5.52 ms to 6.29 ms. In return, the proposed system generated critical and warning events and provided observability through reports and Grafana dashboards.

The defensible conclusion is that rule-based edge processing can improve an IoT smart energy monitoring pipeline by adding timely abnormal-event detection and operational visibility while preserving low-latency ingestion behavior.

# **7 Chapter 7: Conclusion and Future Work**

## **7.1 7.1 Summary**

This thesis designed and implemented an event-driven edge gateway for IoT-based smart energy monitoring. The system uses MQTT for communication, FastAPI for the gateway service, PostgreSQL/TimescaleDB for time-series storage, rule-based processing for abnormal-event detection, and Grafana for observability dashboards.



The problem addressed by the thesis is that raw telemetry pipelines can store measurements but provide limited local intelligence. In such systems, abnormal conditions may only become visible after later analysis or dashboard inspection. The proposed gateway improves this pipeline by validating messages, detecting events at the edge, storing event evidence, and exposing metrics for evaluation.

The evaluation compared a baseline ingestion path with the proposed event-driven gateway. The clean high-throughput A/B test showed that both modes sustained approximately 200 simulator messages per second. The proposed mode increased average telemetry latency from 4.11 ms to 4.43 ms and p99 telemetry latency from 5.52 ms to 6.29 ms. This is a small overhead for the added rule-based event processing.

The anomaly detection experiment showed that the proposed gateway detected overload, power spike, under-voltage, over-voltage, voltage anomaly, and device-failure events. It also rejected malformed payloads and recorded validation failures. These results support the conclusion that rule-based edge processing can add useful event intelligence while preserving low-latency ingestion behavior.

Building on this baseline, Phase 1 added an unsupervised edge Isolation Forest anomaly detector. Evaluated offline on simulator-faithful data, it achieved a precision of 0.61, recall of 0.78, and F1 of 0.69 at its shipped operating point, detecting over-voltage (0.99), power-spike (0.86), and under-voltage (0.83) anomalies well. Its weaker overload recall (0.44) is an informative finding: the simulator's normal load already exceeds the rule's current threshold, so a fixed overload rule is over-sensitive while a distribution-based detector correctly treats those readings as normal.

## 7.2 7.2 Contributions

The project makes the following contributions:

1. It implements an MQTT-based smart energy monitoring pipeline with an edge gateway, time-series storage, and dashboards.
2. It defines a baseline versus proposed comparison for evaluating the cost of event-driven edge processing.
3. It implements validation and data-quality logging for malformed MQTT payloads.
4. It implements configurable rule-based event detection for overload, power spike, voltage, temperature, and device-failure conditions.
5. It stores readings, events, status history, validation logs, and system metrics in PostgreSQL/TimescaleDB.
6. It provides Grafana dashboards for energy overview, device detail, event timeline, system observability, and thesis evaluation.
7. It exports repeatable experimental evidence through scripts and reports.
8. It implements and offline-evaluates an edge Isolation Forest anomaly detector (Phase 1) that scores readings into `model_predictions` and can raise ML events, enabling rules-only, ml-only, and hybrid comparison.
9. It implements a score-gated edge-to-cloud escalation path (Phase 2): an asynchronous forwarder that escalates only flagged readings to a minimal cloud-tier

receiver, with payload bytes counted on both sides so gated forwarding can be compared against an all-to-cloud baseline.

10. It hosts a cloud-side LSTM-autoencoder verifier on the escalated stream (Phase 3) and provides extension points for storage optimization in a later phase.

### 7.3 7.3 Answers to Research Questions

The first research question asked how an event-driven edge gateway can improve smart energy monitoring pipelines. The implemented gateway improves the pipeline by adding validation, event detection, event classification, metrics, and dashboard observability before telemetry becomes only stored history.

The second research question asked how the proposed architecture compares with a baseline telemetry-storage approach. The repeated high-throughput A/B test showed that proposed mode preserved almost the same simulator throughput as baseline mode. The measured latency overhead was small: +0.32 ms average telemetry latency and +0.77 ms p99 telemetry latency.

The third research question asked whether rule-based processing can detect abnormal energy events while maintaining low latency. The anomaly experiment confirmed detection of multiple event types, and telemetry p99 latency remained below 9 ms in that experiment.

The fourth research question asked how ready the platform is for AI/ML extension. Phase 1 answers it concretely: an edge Isolation Forest detector is implemented and offline-evaluated, scoring readings into `model_predictions` and optionally raising ML events through the same path as rules. The platform is no longer ML-ready only in principle — it runs a working edge ML detector, Phase 2 adds a score-gated escalation path to a cloud-tier receiver, and Phase 3 adds a cloud-side LSTM-autoencoder verifier on the escalated stream — while storage optimization is staged as a later phase.

### 7.4 7.4 Limitations

The thesis results should be interpreted within the measured scope.

First, the evaluation used a simulator rather than field hardware. The software pipeline was tested repeatably, but real STM32/ESP hardware behavior, sensor accuracy, Wi-Fi stability, and electrical measurement quality require separate validation.

Second, the experiments were short local Docker runs. They demonstrate controlled behavior but do not prove long-term production reliability, multi-day uptime, cloud deployment readiness, or large-scale field performance.

Third, the ML detector’s reported quality is an offline measurement on simulator-faithful synthetic data; the online A/B measured its operational cost in the live gateway and showed that per-sample inference adds ~12 ms of latency in the current naive implementation (an optimization target, not a fundamental limit). The detector is a single global Isolation Forest; per-device models, batched/offloaded inference, and field validation remain future work.

Fourth, the current results do not prove storage reduction. Proposed mode stores additional event and operational evidence, which increased database growth in the high-throughput experiment. Storage optimization remains future work.

Finally, the system is not a certified commercial metering platform. It is a monitoring and observability architecture, not a billing-grade energy meter.

## 7.5 7.5 ML Roadmap: Completed and Remaining Phases

Phase 1 (completed) implemented unsupervised edge anomaly detection with an Isolation Forest, scoring voltage/current/power readings and storing scores in `model_predictions`. It compared rule-based against model-based detection offline (precision, recall, false positives, per-type recall) and ran an online rules/ml/hybrid A/B (Section 6.7.1) measuring the operational cost — latency, event counts, and storage — of ML scoring in the live gateway.

Phase 2 (completed) added the score-gated escalation path: an asynchronous cloud forwarder in the gateway and a minimal cloud-tier receiver, with a gated-versus-all-to-cloud bandwidth A/B (Sathupadi et al.). The measured run (Section 6.7.2) showed a 53.1% payload reduction under an anomaly-heavy workload with no measurable edge-side latency cost; because gated volume tracks the flag rate, mostly-normal traffic would reduce further, and a periodic-sample channel to preserve cloud-side visibility of normal readings remains future work.

Phase 3 (completed) added the heavier cloud-tier model: an LSTM autoencoder that re-examines escalated readings by reconstruction error and confirms or suppresses each one (Section 6.7.3). Offline, the cloud confirmation step raised two-stage precision from 0.66 to 0.91 by suppressing 643 of 796 edge false positives while dropping only 13 true anomalies; online it confirmed all escalated anomalies at ~0.65 ms per window. The model ships as a numpy artifact so the cloud container needs no deep-learning runtime. It is trained and evaluated on simulator-derived data and treats the window as an unordered batch rather than a true time series, so a genuine sequential model on field telemetry remains future work.

Remaining phases follow the hybrid edge–cloud direction motivated by the literature (Chapter 2.7):

- **Phase 4 — Cloud-tier elastic scaling (Kubernetes).** Deploy the cloud tier on Kubernetes and add horizontal pod autoscaling to the stateless LSTM-AE verifier, then measure how it scales under a variable escalation load (pod count, verification latency, and throughput as the escalated stream grows). The verifier is already a stateless HTTP service with `/health` and a metrics summary endpoint, which makes it a natural autoscaling target. A detailed implementation plan is in `docs/thesis/notes/2026-07-05-phase4-kubernetes-plan.md`. Selective retention, downsampling, and compression are no longer part of the phased roadmap and are now staged as open future work (Section 7.6).
- **Per-device models.** Train a model per device rather than one global model, which the literature shows can improve detection on heterogeneous loads (Mofidul et

al.).

- **Edge inference optimization.** The online A/B showed per-sample scikit-learn scoring adds ~12 ms of latency; batching, offloading to a worker thread, or a lighter/quantized model would reduce this before the detector runs on constrained hardware.

## 7.6 7.6 Future Storage Optimization

The current implementation stores raw readings and event evidence. This is useful for evaluation, but it is not optimized for long-term storage cost. Storage optimization was previously staged as Phase 4 of the roadmap; it has been moved here as open future work so that Phase 4 can instead deliver a measured cloud-tier elastic-scaling result (Section 7.4). Storage optimization remains a valid and independent extension, not a completed result.

Future storage work may include:

- selective raw-reading retention
- downsampling old normal readings
- keeping critical events permanently
- storing event-triggering readings longer than normal readings
- compressing or pruning old operational metrics
- measuring database growth under identical baseline/proposed workloads

A valid storage-reduction claim would require a dedicated implementation and a controlled experiment. It should not be inferred from the current results.

## 7.7 7.7 Future Deployment and Security Work

Future deployment work should address security, reliability, and operational hardening. Practical extensions include:

- TLS-secured MQTT
- per-device credentials
- authentication and authorization for APIs
- production-grade alert delivery
- cloud or hybrid deployment
- backup and restore procedures
- longer-duration soak tests
- hardware-in-the-loop testing

These additions would move the system from a thesis prototype toward a more complete production-ready monitoring platform.

## 7.8 7.8 Final Conclusion

The thesis demonstrates that an event-driven edge gateway can improve an IoT-based smart energy monitoring pipeline by adding validation, rule-based event detection,

event classification, time-series storage, and operational dashboards. The proposed gateway preserved approximately the same simulator throughput as the baseline while adding only a small latency overhead.

The defensible final conclusion is that rule-based edge intelligence can be added to a smart energy monitoring pipeline without sacrificing low-latency ingestion behavior, and that a lightweight unsupervised model can be layered on top at the edge: Phase 1 added an Isolation Forest detector with a measured offline detection quality and an honest account of where rules and models each add value. The system is not yet a production-certified platform, but it now provides a working edge ML foundation and a clear, measurable path toward the hybrid edge–cloud design — cloud-tier models, score-gated escalation, storage optimization, and hardware-field validation.